



Degree Project in Computational Mathematics

Second cycle, 30 credits

Parallel implementation and analysis of the discrete-velocity BGK Boltzmann method

THÉO MONPOUET EKERAM

Parallel implementation and analysis of the discrete-velocity BGK Boltzmann method

THÉO MONPOUET EKERAM

Master's Programme, Applied and Computational Mathematics, 120 credits
Date: June 14, 2025

Supervisors: Pritpal Matharu, Sara Zahedi
Examiner: Sara Zahedi

Swedish title: Parallel implementering och analys av den
diskret-hastighetsbaserade BGK-Boltzmannmetoden

Abstract

This project builds upon the work of Gu, Hu, Matharu et al. (2025), who proved that a local weak solution to the d -dimensional incompressible Navier-Stokes equations can be constructed by taking the hydrodynamic limit of a discrete-velocity Boltzmann equation with a simplified collision operator. Additionally, the convergence rate of this limit was shown numerically to be $\mathcal{O}(\epsilon^2)$ where ϵ is the Knudsen number. This was achieved by implementing the mathematically derived Lattice Boltzmann Method with the lattice structure D2Q9 in two-dimensions and letting the Knudsen number tend to zero while measuring the resulting error compared to a reference solution.

In this thesis, a fast parallel code for the Lattice Boltzmann Method in both two and three dimensions was developed. In addition to D2Q9, the lattice structures D2Q7, D2Q13, and D3Q19 were also implemented. For both the additional lattice structures and the higher dimension, the convergence rate with which the hydrodynamic limit is obtained was investigated by running simulations with decreasing Knudsen numbers and comparing to reference solutions.

The numerical results in this thesis show that the convergence rate is independent of both the lattice structure and the dimension used. Furthermore, the results are consistent with the results by Gu, Hu, Matharu et al. (2025), i.e. the convergence rate matches closely with a power-law relation $a \cdot \epsilon^2$ where a is a constant.

The parallel implementation enables large-scale simulations and speeds up as expected when increasing the number of processors. However, the efficiency of the program decreases with increasing number of processors since more time is spent on communication.

Sammanfattning

Denna studie bygger vidare på arbetet av Gu, Hu, Matharu et al. (2025), som visade att en lokal svag lösning till Navier-Stokes-ekvationerna för okomprimerbara vätskor i d dimensioner kan konstrueras genom att undersöka det hydrodynamiska gränsvärdet av Boltzmann-ekvationen med diskreta hastigheter och en förenklad kollisionoperator. Dessutom visades det numeriskt att konvergensthastigheten för gränsvärdet är $\mathcal{O}(\epsilon^2)$, där ϵ är Knudsen-talet. Detta gjordes genom att implementera den matematiskt härledda Lattice Boltzmann-metoden med latticestrukturen D2Q9 i två dimensioner, och låta Knudsen-talet gå mot noll samtidigt som felet jämfört med en referenslösning undersöktes.

I detta projekt utvecklades en snabb parallell implementering av Lattice-Boltzmann metoden för att kunna köra storskaliga simuleringar i både två och tre dimensioner med de nya latticestrukturena D2Q7, D2Q13, och D3Q19. Konvergensthastigheten för både de nya latticestrukturena och den nya dimensionen undersöktes genom att köra simulationer med minskande Knudsen-tal och jämföra med referenslösningar.

Resultatet visar att felet kan approximeras med sambandet $a \cdot \epsilon^2$ där a är en konstant, vilket motsvarar att konvergensthastigheten $\mathcal{O}(\epsilon^2)$ oberoende av både latticestrukturen och dimension, vilket är samma resultat som Gu, Hu, Matharu et al. (2025) uppnådde för D2Q9 i två dimensioner.

Körningshastigheten av den parallella implementeringen är snabb nog att kunna köra storskaliga simulationer och blir snabbare som väntat vid användning av fler processorer. Programmets effektivitet minskar dock med fler processorer, eftersom mer tid går åt till kommunikation.

Acknowledgments

I would like to express my gratitude to everyone who has supported me throughout this thesis project.

First, I want to thank my supervisor Pritpal Matharu for our weekly meetings and his guidance during the project, his knowledge and encouragement have been essential to the success of this thesis.

Second, I want to thank my supervisor and examiner Sara Zahedi for her feedback on this report.

Finally, I want to thank my friends and family for their support and for reminding me to take breaks.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
2	Background	3
2.1	The Lattice Boltzmann Method	3
2.1.1	The Boltzmann equation	3
2.1.2	Lattice structures	4
2.1.3	The hydrodynamic limit	5
2.1.4	PDE setup	5
2.2	Pseudo-spectral methods	6
3	Numerical Method	8
3.1	Pseudo-spectral method	8
3.1.1	Mathematical formulation	8
3.1.2	Time integration	9
3.1.3	Dealiasing	10
3.2	Higher dimensions	11
3.3	Parallelization approach	12
3.4	Hardware and Software	13
4	Numerical setup	14
4.1	Lattice structures	14
4.2	Initial conditions	15
4.3	Validation and visualization	16
4.4	Algorithm	17
5	Results	19
5.1	Visualization of results	19
5.2	Hydrodynamic limit	22
5.2.1	Lattice modification	22
5.2.2	Turbulent simulation	24
5.2.3	Higher dimensions	24
5.3	Parallel performance	25
6	Discussion	28
6.1	Lattice modification	28
6.2	Turbulent simulation	28
6.3	Higher dimensions	28
6.4	Running time	28
7	Conclusion and future work	30
7.1	Future work	30
	References	31
A	Lattice structure visualization	33
B	Vorticity description	34
B.1	From velocity to vorticity field	34
B.2	From vorticity to velocity field	34
C	Navier-Stokes in vorticity form	35
D	Orthogonality condition for PS method	36

E Slices of the three-dimensional vector field	38
F Parallel performance tables	40

1 Introduction

1.1 Motivation

The study of fluid dynamics plays an important role in understanding, analyzing and predicting the behavior of complex systems in many fields. Examples include modeling airflow over airplane wings to accurately predict drag- and lift coefficients, and modeling of blood flow in vital organs to improve diagnostics and treatment strategies.

From Versteeg and Malalasekera [21], computational fluid dynamics (CFD) is the analysis of systems involving fluid flow, heat transfer and associated phenomena, by means of computer-based simulation. By this definition, CFD started being utilized as a tool dating back to the 1960s. In that time, larger and faster military aircraft were being developed, and the wind tunnel facilities needed to test these aircraft concepts in all relevant conditions did not exist. CFD, although then largely limited to two-dimensional flow problems, became a viable complement to the wind tunnel tests. During the end of the 1980s, faster computers and numerical acceleration techniques started to get developed, which enabled the study of small-scale turbulent flows.

The analysis of fluid flow began even earlier than that, with an example being the work in Richardson [18] in 1922, which used human computers to make the calculations by hand or calculator. This was a very slow method, taking weeks to simulate a short time.

To model the fluid dynamics of incompressible fluids, the standard framework is usually to numerically solve the incompressible Navier-Stokes (NS) equations, which describe conservation of mass and momentum for viscous fluids. In the space-time domain $(\mathbf{x}, t) \in \Omega \times (0, T]$ where $\Omega = \mathbb{T}^d$ ($d = 2$ or 3) is the physical domain defined as a d -dimensional torus with periodic boundary conditions, and $\mathcal{T}$ is some final time, these can be written as

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} + \nabla p - \nu \Delta \mathbf{u} = \mathbf{f} & \text{in } (0, \mathcal{T}] \times \Omega, \\ \nabla \cdot \mathbf{u} = 0 & \text{in } (0, \mathcal{T}] \times \Omega, \\ \mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0 & \text{in } \Omega, \end{cases} \quad (1)$$

where $\mathbf{u}(\mathbf{x}, t)$ is the velocity field at position $\mathbf{x}$ and time t , p is the pressure, $\mathbf{f}$ is an external force, ν is the kinematic viscosity, and $\mathbf{u}_0$ represents the initial condition.

In the context of deriving analytical solutions to (1), the concept of a weak local solution is useful. A weak solution to a partial differential equation can include singularities, i.e. that the solution may not be continuously differentiable. A local weak solution is a weak solution that is only guaranteed to exist on a short time interval, as described in Evans [6].

Numerically solving the Navier-Stokes equations with methods such as finite elements, finite volumes or finite differences, is a macroscopic approach, meaning that the method describes the average behavior of the velocity and pressure quantities in a given scale. In macroscopic approaches, molecular interactions are not explicitly considered, which means that for very small scales, solving the Navier-Stokes equation can fail to capture the flow accurately. On the other hand, numerical methods for microscopic models which simulate every particle and its interactions with every other particle would capture interactions on the smallest scales. The problem is that it would be unfeasible to compute all the interactions needed to model fluid flow at a larger scale since the computations needed would be too expensive. As a middle ground, the mesoscopic approach models fluid flow by considering distributions or representative collections of particles, which means that numerical results can capture microscale behavior while still being fast enough to be used in larger-scale simulations [12].

1.2 Objectives

This project is built upon the work by Gu et al. [10], who proved that a local weak solution to the d -dimensional incompressible Navier-Stokes equations can be constructed as the hydrodynamic limit of the discrete-velocity Boltzmann equation with a simplified Bhatnagar-Gross-Krook collision operator. They also implemented the discrete-velocity Lattice Boltzmann method in two dimensions and investigated the rate at which the hydrodynamic limit is achieved for the lattice structure D2Q9. The objective of this project is to extend this to different lattices and to both two- and three dimensional cases by implementing a faster and parallel code for the Lattice Boltzmann method and investigating the rate at which the hydrodynamic limit is achieved for these new cases.

Excluding the appendices, this thesis is organized into seven sections, structured as follows:

Section 2 introduces the Lattice Boltzmann equation and gives background to the methods used, Section 3 covers the mathematical and numerical methods that were used. Section 4 explains the numerical setup of the simulations done. Section 5 presents the numerical results, which are then discussed in Section 6. Lastly, Section 7 offers conclusions as well as insights for future work.

2 Background

This section provides a background to the Lattice Boltzmann method (in Section 2.1), including the Boltzmann equation and its simplifications in Section 2.1.1, as well as the definition of lattice structures in Section 2.1.2. A definition of the hydrodynamic limit is given in Section 2.1.3, and the differential equation setup is described in Section 2.1.4. In Section 2.2, the pseudo-spectral method is introduced.

2.1 The Lattice Boltzmann Method

Solving the equations of fluid dynamics, particularly the Navier-Stokes equations (1), is a longstanding challenge and remains unsolved in the general case. From Galdi [9], a factor contributing to the analytical difficulties is that the convective term $\mathbf{u} \cdot \nabla \mathbf{u}$ makes the system nonlinear. The coupled nature of the velocity and pressure fields, and the full three-dimensionality of the problem also contribute to the fact that the Navier-Stokes equations are mentioned in Fefferman [7] as one of the Millennium unsolved problem.

To understand problems with complex geometries or non periodic boundary conditions, numerical methods must therefore be used. From Krüger et al. [12], numerical Navier-Stokes solvers can be difficult to implement correctly, and are often not ideal for parallelizing due to for example having to solve the Poisson equation, which is a global operation in the sense that a change in pressure at one point affects the pressure in all other points.

The Lattice Boltzmann method (LBM) is a numerical approach for simulating transport problems in fluid dynamics, thermal dynamics, and crowd dynamics. The first forms of the LBM were in the context of gas automata, where particles were existing on a square lattice in which they would move and collide with other particles respecting the laws of mass- and momentum conservation, see Krüger et al. [12].

In the case of simulating fluid dynamics, the LBM is based on kinetic theory rather than directly solving the Navier-Stokes equations, meaning that the method describes the statistical behavior of the particle system in space and time. This method comprises of the space and time- discretized version of the Boltzmann equation along with a lattice structure for discretizing the velocity space. Contrary to traditional Navier-Stokes solvers, the LBM is relatively simple to implement and parallelize. The reason is, as described in Krüger et al. [12], that the Boltzmann equation is a simple hyperbolic equation and that the collision terms only depend on the objective variables and not its gradients. This means that each processor stores and operates on a distinct part of the data and no complicated communication is needed as with partial differential equations with global coupling similar to the Navier-Stokes system.

2.1.1 The Boltzmann equation

The basis behind the LBM is the Boltzmann transport equation, which describes the statistical behavior of a system of particles. As described in Krüger et al. [12], the Boltzmann transport equation can be written as:

$$\frac{\partial f}{\partial t} + \mathbf{v} \cdot \nabla f + \mathbf{F} \cdot \frac{\partial f}{\partial \mathbf{v}} = \left(\frac{\partial f}{\partial t} \right)_{collision}, \quad (2)$$

where f is the probability distribution function, and $\mathbf{v}$ are the infinitely many possible velocities. The second term represents the advection term (the free streaming of particles), the third term accounts for external forces perturbing the system, and the right hand side is the collision term modeling particle interaction.

To obtain a numerical method for (2) which can easily be used in computational fluid dynamics, the same simplifications as described in Gu et al. [10] are made:

1. Instead of considering an infinite number of velocity vectors $\mathbf{v}$, a finite set of discrete velocities v_i which must themselves be finite, are considered. Under specific conditions, this set along with a corresponding set of weights is called an isotropic lattice. This will be explained further in Section 2.1.2.
2. Like most numerical solvers used in computational fluid dynamics, the time and physical space are discretized into small uniform time steps Δt and $\Delta \mathbf{x} = (\Delta x, \Delta y, \Delta z)$ (for a three-dimensional simulation).
3. The full Boltzmann collision operator is usually too complex and expensive to compute for regular use. Instead, a simplified version called the Bhatnagar-Gross-Krook (BGK) operator is regularly used. It is, in its discretized version, given by:

$$\Omega_i^{\text{collision}}(f) = \frac{\Delta t}{\tau_R}(f_i^{\text{eq}} - f_i), \quad (3)$$

where τ_R represents the relaxation time and f_i^{eq} is the i -th equilibrium distribution.

Taking these three simplifications into account as well as neglecting the term representing the external forces (the third term in (2)), the Lattice Boltzmann equation (LBE) can be defined as

$$f_i(\mathbf{x} + v_i \Delta \mathbf{x}, t + \Delta t) = f_i(\mathbf{x}, t) + \Omega_i^{\text{collision}}(\mathbf{x}, t), \quad i = 0, 1, \dots, n,$$

where v_i are the finitely many velocity vectors chosen, f_i is the particle velocity distribution function along v_i , Ω_i is the BGK operator, and n represents the number of non-zero particle velocities that are considered. The Lattice Boltzmann method involves solving the Lattice Boltzmann equation through numerical methods.

2.1.2 Lattice structures

For discretizing the space of possible velocities, different lattice structures can be used. These structures, as mathematically described in Gu et al. [10], follow the definition:

Let d be the spatial dimension that the lattice structure is defined in. Let $n \in \mathbb{N}$ be the number of non-zero velocities that the structure should contain, and let $\mathcal{V} := \{\mathbf{v}_0 = \mathbf{0}, \mathbf{v}_1, \dots, \mathbf{v}_n\} \subset \mathbb{R}^d$ be the set of unique velocities in the structure. Let $w = (w_0, \dots, w_n) \in \mathbb{R}_+^{n+1}$ be a set of weights such that each velocity vector $\mathbf{v}_i$ is paired with a weight w_i . The pair $(\mathcal{V}, w)$ is called a lattice.

For a lattice structure to be usable when dealing with computational fluid dynamics, it usually has to follow a set of rules. These rules concern among other things the symmetry of the lattice. If the structure is not symmetric, the solution would depend on the grid orientation, which is not desirable in most cases. A lattice $(\mathcal{V}, w)$ is called an isotropic lattice associated with the speed of sound $c_s > 0$ if it satisfies

$$\left\{ \begin{array}{l} \sum_{i=0}^n w_i = 1, \\ \sum_{i=0}^n w_i v_{i,\alpha} = 0, \\ \sum_{i=0}^n w_i v_{i,\alpha} v_{i,\beta} = c_s^2 \delta_{\alpha\beta}, \\ \sum_{i=0}^n w_i v_{i,\alpha} v_{i,\beta} v_{i,\gamma} = 0, \\ \sum_{i=0}^n w_i v_{i,\alpha} v_{i,\beta} v_{i,\gamma} v_{i,\zeta} = c_s^4 (\delta_{\alpha\beta} \delta_{\gamma\zeta} + \delta_{\alpha\gamma} \delta_{\beta\zeta} + \delta_{\alpha\zeta} \delta_{\beta\gamma}), \end{array} \right. \quad (4)$$

for all possible permutations of $\alpha, \beta, \gamma, \zeta$ where $1 \leq \alpha, \beta, \gamma, \zeta \leq d$. Throughout the report, the notation $v_{i,\eta}$ represents the η -component of the velocity vector $\mathbf{v}_i$, where $1 \leq \eta \leq d$, and δ is defined as

$$\delta_{ab} = \begin{cases} 1, & a = b \\ 0, & \text{else} \end{cases},$$

for all constants a, b . To differentiate between different lattices in this report, the same notation as Gu et al. [10] will be used, meaning that a lattice consisting of n velocity vectors in d dimensions will have the notation $\mathbf{DdQn}$.

2.1.3 The hydrodynamic limit

From Krüger et al. [12], the hydrodynamic limit represents a transition from microscopic particle-based systems to macroscopic fluid-like behavior. To characterize this transition, the Knudsen number is used, denoted in this report by ϵ , and defined as

$$\epsilon = \frac{\ell_{mfp}}{\ell},$$

where ℓ_{mfp} represents the mean free path, i.e. the average distance a particle travels between collisions, and ℓ is the physical length scale to make ϵ dimensionless. It can be seen from this that the lower ϵ is, the more refined the method gets, since the collisions happen more often. For $\epsilon \ll 1$, the solution to the LBE should be close enough to the macroscopic solution to be considered valid. For $\epsilon > 1$, the solution does not match well enough to draw any conclusions. Having ϵ as a parameter to the simulation and letting $\epsilon \rightarrow 0$ corresponds to having the spacing between individual particles tending to zero.

2.1.4 PDE setup

According to Saint-Raymond [19] and similarly to Gu et al. [10], Equation (2) along with the BGK operator (3) can be combined, in its discretized velocity form, to the following partial differential equation (PDE):

$$\epsilon \partial_t g_i^\epsilon + \mathbf{v}_i \cdot \nabla_x g_i^\epsilon = \frac{1}{\epsilon \nu} (g_{i,eq}^\epsilon - g_i^\epsilon), \quad g_i^\epsilon|_{t=0} = g_i^{0,\epsilon}, \quad (5)$$

where

$$g_{i,eq}^\epsilon := \rho^\epsilon + \frac{\mathbf{v}_i \cdot \mathbf{u}^\epsilon}{c_s^2} + \frac{\epsilon}{2c_s^4} \sum_{\alpha=1}^d \sum_{\beta=1}^d u_\alpha^\epsilon u_\beta^\epsilon (v_{i,\alpha} v_{i,\beta} - c_s^2 \delta_{\alpha\beta}), \quad (6)$$

and

$$\rho^\epsilon := \sum_{i=0}^n w_i g_i^\epsilon, \quad u^\epsilon := \sum_{i=0}^n w_i \mathbf{v}_i g_i^\epsilon, \quad (7)$$

where $g_i^\epsilon = g_i^\epsilon(\mathbf{x}, t)$, $i = 0, \dots, n$ are the probability density functions corresponding to the velocity $\mathbf{v}_i$ and weight w_i associated with the chosen lattice structure. Here $\epsilon > 0$ is the Knudsen number, d is the chosen dimension, ν is the relaxation time, ρ^ϵ is the density scalar field, and u^ϵ is the velocity vector field.

This is the PDE that will be solved for different lattices, dimensions, Knudsen numbers, and relaxation times. Relaxation time in the context of the Lattice Boltzmann method is similar to the viscosity magnitude in regular fluid flow in the sense that lower relaxation time leads to a faster-flowing fluid. The solution to (5) at time t is the probability distribution function for all lattice directions and depends on both space and time, $g_i^\epsilon(\mathbf{x}, t)$ for $i = 0, \dots, n$. From that solution, interesting quantities such as velocity and density fields can be recovered using Equation (7).

For two dimension, the domain that (5) is solved over is denoted by $\mathbb{T}_x^2$ and is the periodic spatial domain $[0, 1] \times [0, 1]$ (also called a 2D torus). Similarly for three dimensions, the domain is denoted by $\mathbb{T}_x^3$ and is $[0, 1] \times [0, 1] \times [0, 1]$ and periodic in all directions.

2.2 Pseudo-spectral methods

The pseudo spectral method is a numerical technique used in solving partial differential equations (PDEs), particularly in fluid dynamics and kinetic theory. The main idea is to transform the differential operators into algebraic ones using a spectral representation of the equations.

As described in Boyd [2], the purely spectral method approximates a function using a series expansion of basis functions. Usually, Fourier basis functions are used for periodic problems, and Chebyshev polynomials for non-periodic problems. In contrast to finite difference- or element methods, which create local approximation of the derivative terms, spectral methods utilize global representations, offering higher accuracy with fewer grid points.

Using this method, differentiation is taking place in Fourier space. This is because of the fact that differentiation is a simple algebraic operation in Fourier space, in contrast to the physical space where an approximation like finite differences may be used. Specifically, taking the derivative of a function corresponds to multiplying its Fourier coefficients by ik where k is the wave number (this process is explained in Section 3.1.1). However, one of the biggest problems with purely spectral methods is the need to evaluate derivatives of non-linear terms in the transformed space, which can be computationally very expensive as it would involve global convolutions.

One solution to this problem, according to Boyd [2], is to use the pseudo-spectral method. Similar to the spectral method, it performs the differentiation operations in Fourier (or Chebyshev) space, but evaluates nonlinear terms in physical space. This approach is usually implemented using the following steps:

1. Differentiation is performed in spectral space.
2. The function is converted to physical space.
3. Nonlinear terms are evaluated in physical space.
4. The function is transformed back to spectral space.

Since methods like the LBM are time-dependent, these steps are repeated many times. This back and forth between physical and spectral space can be computationally expensive, but is often better than having to compute the convolutions that would

emerge if the non-linearities are handled in spectral space. This will be explained in detail in Section 3.3.

Although the pseudo-spectral method has good accuracy and a good computational efficiency, it can suffer from problems such as aliasing errors. This can be helped with dealiasing methods, which will be addressed in Section 3.1.3.

3 Numerical Method

This section describes the methods used in this project. Section 3.1 describes how to setup the pseudo-spectral method, including the underlying mathematics, time stepping methods, and ways to handle aliasing errors. Section 3.2 describes what needs to be changed when going from two dimensions to three, and Section 3.3 describes the method behind parallelizing the algorithm. Lastly, Section 3.4 describes the hardware and software used in this project.

3.1 Pseudo-spectral method

3.1.1 Mathematical formulation

The basis of formulating a spectral method with Fourier bases is to approximate the objective function using a truncated Fourier series. Let $\mathcal{K} = \{\mathbf{k} \in \mathbb{Z}^d : |\mathbf{k}| \leq K\}$ where K is the maximum resolved wavenumber. Define the Galerkin approximation of g_i^ϵ in d dimensions at $\mathbf{x}$ using the Fourier coefficient $\hat{g}_{i,\mathbf{k}}^\epsilon$ as

$$g_{i,K}^\epsilon(\mathbf{x}, t) := \sum_{\mathbf{k} \in \mathcal{K}} \hat{g}_{i,\mathbf{k}}^\epsilon(t) e^{\imath \mathbf{k} \cdot \mathbf{x}}, \quad (8)$$

where $\mathbf{k} = (k_x, k_y)$ in two dimensions and $\mathbf{k} = (k_x, k_y, k_z)$ in three dimensions. In this report, $\imath$ refers to the imaginary unit, distinct from the indexing variable i used when referring to the different equations in the system (5). Depending on the dimension used, the computational domain is bounded by $[L_x, L_y, (L_z)]$ and is discretized by $N_x, N_y, (N_z)$ grid points in each respective direction. The grid points $\mathbf{x} = (x, y, (z))$ are uniformly placed in the domain according to

$$x_p = \frac{pL_x}{N_x}, \quad y_q = \frac{qL_y}{N_y}, \quad z_l = \frac{lL_z}{N_z},$$

with $p = 0, 1, \dots, N_x - 1$, $q = 0, 1, \dots, N_y - 1$, and $l = 0, 1, \dots, N_z - 1$. There is no need to include the last grid point since the problem is periodic and the Fourier bases are periodic by nature, i.e. $x_0 = x_{N_x}$, $y_0 = y_{N_y}$ and $z_0 = z_{N_z}$.

The goal is to input the truncated Fourier series representation of the objective function into the PDE (5). To do that however, the time and spatial derivatives need to be calculated. The time derivative is the most straightforward, since the only term in (8) that is time dependent is the Fourier coefficient. The first order spatial derivatives only have an extra term coming from differentiating the exponential term. This process gives:

$$\begin{aligned} \frac{\partial g_{i,K}^\epsilon}{\partial t}(\mathbf{x}, t) &= \sum_{\mathbf{k} \in \mathcal{K}} \frac{d}{dt} \hat{g}_{i,\mathbf{k}}^\epsilon(t) e^{\imath \mathbf{k} \cdot \mathbf{x}}, \\ \frac{\partial g_{i,K}^\epsilon}{\partial x}(\mathbf{x}, t) &= \sum_{\mathbf{k} \in \mathcal{K}} (\imath k_x) \hat{g}_{i,\mathbf{k}}^\epsilon(t) e^{\imath \mathbf{k} \cdot \mathbf{x}}, \\ \frac{\partial g_{i,K}^\epsilon}{\partial y}(\mathbf{x}, t) &= \sum_{\mathbf{k} \in \mathcal{K}} (\imath k_y) \hat{g}_{i,\mathbf{k}}^\epsilon(t) e^{\imath \mathbf{k} \cdot \mathbf{x}}, \\ \frac{\partial g_{i,K}^\epsilon}{\partial z}(\mathbf{x}, t) &= \sum_{\mathbf{k} \in \mathcal{K}} (\imath k_z) \hat{g}_{i,\mathbf{k}}^\epsilon(t) e^{\imath \mathbf{k} \cdot \mathbf{x}}. \end{aligned}$$

Substituting these series representations into (5) and applying orthogonality (see the explanation in Appendix D) separately for every equation in the system, we get the

following new system of differential equations:

$$\epsilon \partial_t \hat{g}_{i,\mathbf{k}}^\epsilon + \mathbf{v}_i \cdot \mathbf{k} \cdot \hat{g}_{i,\mathbf{k}}^\epsilon = \frac{1}{\epsilon \nu} (\hat{g}_{i,eq,\mathbf{k}}^\epsilon - \hat{g}_{i,\mathbf{k}}^\epsilon), \quad i = 0, \dots, n, \quad (9)$$

where

$$\hat{g}_{i,eq,\mathbf{k}}^\epsilon = \hat{\rho}_\mathbf{k}^\epsilon + \frac{\mathbf{v}_i \cdot \hat{\mathbf{u}}_\mathbf{k}^\epsilon}{c_s^2} + \frac{\epsilon}{2c_s^4} \sum_{\alpha,\beta=1}^d \widehat{(u_\alpha^\epsilon u_\beta^\epsilon)}_\mathbf{k} (v_{i,\alpha} v_{i,\beta} - c_s^2 \delta_{\alpha\beta}), \quad (10)$$

or, in a form that will be easier to handle when time stepping:

$$\partial_t \hat{g}_{i,\mathbf{k}}^\epsilon = \frac{1}{\epsilon^2 \nu} (\hat{g}_{i,eq,\mathbf{k}}^\epsilon - \hat{g}_{i,\mathbf{k}}^\epsilon) - \frac{\mathbf{v}_i \cdot \mathbf{k} \cdot \hat{g}_{i,\mathbf{k}}^\epsilon}{\epsilon}, \quad i = 0, \dots, n, \quad (11)$$

where $\hat{\rho}_\mathbf{k}^\epsilon$ and $\hat{\mathbf{u}}_\mathbf{k}^\epsilon$ are the Fourier coefficients of ρ^ϵ and $\mathbf{u}^\epsilon$.

The difficulty of using spectral method is that there exists a non-linear term in (10). More specifically, the term $u_\alpha u_\beta$ is a multiplication involving the objective function g_i^ϵ . Since a multiplication in physical space means a convolution sum in Fourier space, the algorithm would need to compute an convolution in order to calculate $\hat{g}_{i,eq,\mathbf{k}}^\epsilon$, which would have the time complexity $\mathcal{O}(N^{2d})$ where N is the spatial resolution assuming that $N = N_x = N_y = N_z$, and d is the dimension number, as seen in Canuto et al. [3].

The solution is to use a pseudo-spectral approach, transforming $\hat{\mathbf{g}}_i$ back to physical space, evaluating the multiplications needed, and then transforming back to Fourier space. From FFTW docs [8], transforming back and forth between physical and Fourier space can be done with the d -dimensional Fast Fourier Transform in $\mathcal{O}(N^d \log N)$ time. This is a large improvement from the $\mathcal{O}(N^{2d})$ computational cost of computing the convolution directly.

Using this method, the ordinary differential Equation (11) can be time stepped using a standard time integration method to get the solution in Fourier space.

3.1.2 Time integration

Since Equation (11) is a system of coupled first order ordinary differential equations, a standard fourth order Runge-Kutta method (RK4) can be used as described in Sauer [20]. In this project, the RK4 method is used mainly for its fourth-order accuracy. This is because when investigating the convergence of the solution when approaching the hydrodynamic limit, the errors arising from the time stepping itself should be kept minimal to not influence the results. This is done with the trade-off of having a higher computational cost per time step compared to, for example, the forwards Euler method or the Leap-frog method.

For ease of notation, let $\hat{\mathbf{g}}_\mathbf{k}^\epsilon = (\hat{g}_{0,\mathbf{k}}^\epsilon, \hat{g}_{1,\mathbf{k}}^\epsilon, \dots, \hat{g}_{n,\mathbf{k}}^\epsilon)$, $\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon = (\hat{g}_{0,eq,\mathbf{k}}^\epsilon, \hat{g}_{1,eq,\mathbf{k}}^\epsilon, \dots, \hat{g}_{n,eq,\mathbf{k}}^\epsilon)$, and $\mathbf{v} = (v_0, v_1, \dots, v_n)$. The RK4 method can then be formulated as follows:

Let the system of ODEs to be solved be defined as

$$\partial_t \hat{\mathbf{g}}_\mathbf{k}^\epsilon = \mathbf{f}(t, \hat{\mathbf{g}}_\mathbf{k}^\epsilon),$$

for some vector valued function $\mathbf{f}$. Discretizing the time interval $(0, \mathcal{T}]$ uniformly with a timestep Δt , where $\mathcal{T}$ is the final time, and denoting by $\hat{\mathbf{g}}_\mathbf{k}^{\epsilon,m}$ the approximate solution at time $t = t_m$, we solve the following equations for $\hat{\mathbf{g}}_\mathbf{k}^{\epsilon,m+1}$, $m = 1, 2, 3, \dots$, with $\hat{\mathbf{g}}_\mathbf{k}^{\epsilon,0}$ (the initial solution) given:

$$\begin{aligned}
s_1^m &= \mathbf{f}(t_m, \hat{\mathbf{g}}_k^{\epsilon, m}), \\
s_2^m &= \mathbf{f}\left(t_m + \frac{\Delta t}{2}, \hat{\mathbf{g}}_k^{\epsilon, m} + \frac{\Delta t}{2} s_1^m\right), \\
s_3^m &= \mathbf{f}\left(t_m + \frac{\Delta t}{2}, \hat{\mathbf{g}}_k^{\epsilon, m} + \frac{\Delta t}{2} s_2^m\right), \\
s_4^m &= \mathbf{f}(t_m + \Delta t, \hat{\mathbf{g}}_k^{\epsilon, m} + \Delta t s_3^m), \\
\hat{\mathbf{g}}_k^{\epsilon, m+1} &= \hat{\mathbf{g}}_k^{\epsilon, m} + \frac{\Delta t}{6}(s_1^m + 2s_2^m + 2s_3^m + s_4^m).
\end{aligned}$$

Applying this method to Equation (9), the function $\mathbf{f}(t, \hat{\mathbf{g}}_k^\epsilon)$ can be identified as

$$\mathbf{f}(t, \hat{\mathbf{g}}_k^\epsilon) = \mathbf{f}(\hat{\mathbf{g}}_k^\epsilon) = \frac{1}{\epsilon^2 \nu} (\hat{\mathbf{g}}_{eq, k}^\epsilon - \hat{\mathbf{g}}_k^\epsilon) - \frac{\mathbf{v} \cdot \nu \mathbf{k} \cdot \hat{\mathbf{g}}_k^\epsilon}{\epsilon}.$$

3.1.3 Dealiasing

A source of error when it comes to pseudo-spectral methods is aliasing. Figure 3.1 showcases this problem: sampling a 15Hz sine wave with a sampling rate of 14Hz clearly can be interpreted as a lower frequency wave, meaning that we do not know what the underlying frequency is. This type of error is called temporal aliasing since the solution depends strongly on which time instances the sampling is done at.

The aliasing errors arising from pseudo spectral methods are similar in nature. Suppose that $u(x)$ and $v(x)$ are functions on a periodic domain. The approximations of u and v containing the modes up to m are defined as

$$u_K = \sum_{k=-m}^m \hat{u}_k e^{ikx}, \quad v_K = \sum_{k=-m}^m \hat{v}_k e^{ikx}.$$

Aliasing occurs when there is a non-linear operation in physical space such as $u(x)v(x)$. The product of the two functions, $w(x)$ can be obtained by multiplying the respective series:

$$w(x) = u(x)v(x) = \left(\sum_{k=-m}^m \hat{u}_k e^{ikx} \right) \left(\sum_{k=-m}^m \hat{v}_k e^{ikx} \right) = \sum_{k=-2m}^{2m} \hat{w}_k e^{ikx}.$$

The higher frequency waves cannot be represented with the discrete grid used for the function u and v . These unresolved high frequencies will therefore contribute to the aliasing error.

To circumvent this problem, solutions like the 3/2 rule explained in Dutykh [5] can be used to cut off higher frequencies. In this case, a cutoff function defined as

$$\rho(k/N) = \begin{cases} 1, & |k/N| \leq 2/3, \\ 0, & \text{else,} \end{cases} \quad (12)$$

where N is the spatial resolution. However, Hou and Li [11] proposed a exponential dealiasing filter to minimize the possible problems with a hard 3/2 cut-off rule such as Gibbs phenomenon, which can make the solution oscillate near sharp edges. This exponential cutoff function is defined as

$$\rho(k/N) = e^{-\alpha(|k|/N)^m}, \quad (13)$$

where $\alpha, m = 36$ as explained in [11]. In Figure 3.2, the two cutoff functions are shown.

The exponential filter (13) keeps around 12% – 15% more modes than the cutoff function (12) and performed better when looking at minimizing errors due to aliasing. It can also easily be generalized to higher dimensions by considering the modes k_x, k_y, k_z separately. This is the dealiasing function used in this project.

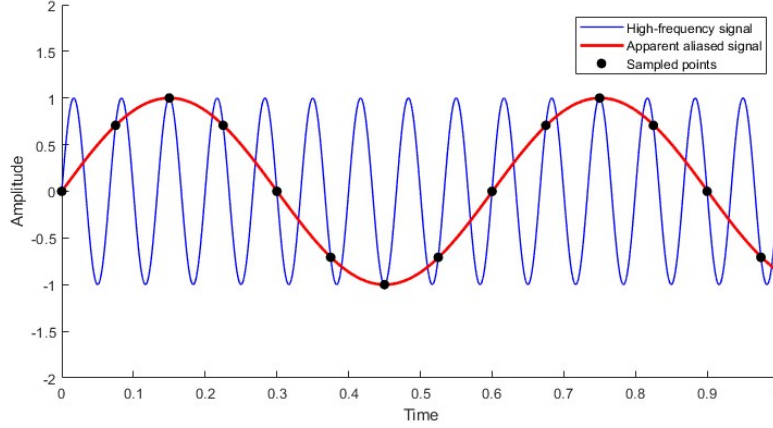


Figure 3.1: Visual explanation of the temporal aliasing effect, where sampling a high frequency wave with a too low sampling rate could be interpreted as a lower frequency wave.

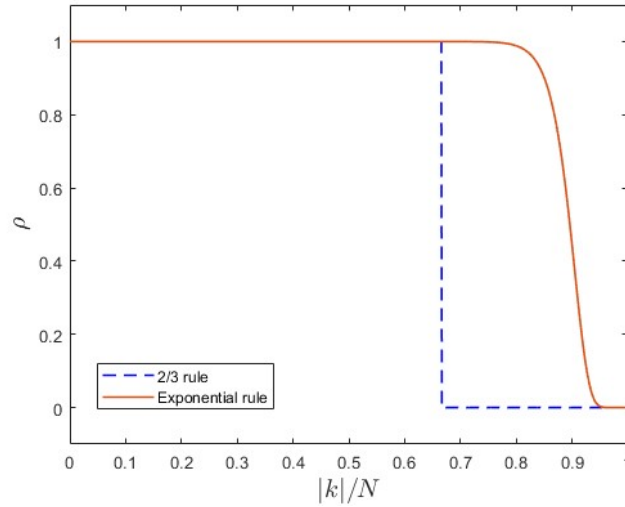


Figure 3.2: Plot of the two cutoff functions (12) and (13) showcasing the difference in which frequencies they cut off.

3.2 Higher dimensions

Generalizing the problem to three dimensions is almost trivial compared to traditional Navier-Stokes solvers. Equations (5)-(7) still hold with $d = 3$. The differences in the algorithm itself are limited to using the three-dimensional fast Fourier transform and accounting for the extra dimension in the time-stepping method. The latter still follows the notation in Section 3.1.2.

The three dimensional problem is much more computationally heavy than its two dimensional variant. For example, a two dimensional spatial discretization of 128×128 grid points results in $128^2 = 16384$ elements for the Fourier transform to handle. If the same spatial resolution is needed in three dimension, it would result in $128^3 = 2097152$ elements. Thus, to be able to run simulations in three dimension, a parallel approach is clearly needed. This will be further discussed in Section 3.3.

3.3 Parallelization approach

Parallelization of the Lattice Boltzmann method, especially when combined with a Fourier based pseudospectral method, allows us to run larger simulations without it taking an unreasonable amount of time.

The parallelization itself is almost trivial since the LBM and the time stepping scheme only does element-wise operations. That means that the physical space can be divided between processors. More specifically, if a $[N \times N]$ is used in the physical space, each processor would store and handle operations on a $[N/P \times N]$ grid where P is the number of processes. In three dimensions, having a $[N \times N \times N]$ grid would result in each process handling $[N/P \times N \times N]$ elements. Here, P and N are always chosen so that P divides N . As Wilkinson and Allen [22] describe, there are ways to handle parallelization without having that criteria. Such methods utilize load balancing to distribute tasks (in this case grid elements) equally to the processes. For simplicity, these methods will not be implemented in this project.

Not every part of the method is trivial to parallelize. An important computation in the method that requires communication between processes is the two (and three)-dimensional Fourier transform. As described in Chu and George [4], this procedure involves making a row-wise Fourier transform, transposing the matrix to make another row-wise transform, followed by another transposition to restore the original structure. These transposition operations are the most expensive when it comes to communication. However, the final transpose operation can be omitted at the expense of obtaining a transposed matrix in Fourier space.

Another possible optimization is the use of conjugate symmetry in Fourier space for real valued functions. Since the forward Fourier transform is symmetric in the sense that $\hat{f}_k = \hat{f}_{N-k}$ where $\hat{f}_k$ is the Fourier coefficient of the function f corresponding to the wavenumber k , and N is the maximum wavenumber. This means that only half of the Fourier coefficients need to be stored and operated on, since the other half always can be reconstructed.

To achieve both these optimizations, a C++ library called "Fastest Fourier Transform In The West" ([8]) is used in combination with the Message Passing Interface (MPI). This library implements highly efficient routines for multidimensional fast Fourier transforms, additionally having flags to support the removal of the last transposition step and utilize the conjugate symmetry. The specific routines used for forward and inverse Fourier transforms in this project are `fftw_mpi_plan_dft_r2c_2d` and `fftw_mpi_plan_dft_c2r_2d` for the two-dimensional case, and `fftw_mpi_plan_dft_r2c_3d` and `fftw_mpi_plan_dft_c2r_3d` for three dimensions along with the flags `FFTW_MPI_TRANSPOSED_OUT` and `FFTW_MPI_TRANSPOSED_IN` for omitting the last transposition step.

To measure the quality of a parallel algorithm, two factors are important to look at: the parallel speedup factor and the parallel efficiency. The speedup factor as a function of the number of processors used, is defined as

$$S(P) = \frac{t_s}{t_P},$$

where t_s refers to the algorithm's execution time using a single processor system (with the best sequential algorithm), and t_P refers to the algorithm's execution time using P

processors. When the sequential computation can be divided into equal-duration parts and no additional overhead such as communication time between processors is necessary, the maximum value expected for $S(P)$ is

$$S(P) = \frac{t_s}{t_s/P} = P. \quad (14)$$

Since communication between processors is needed in this implementation, it is interesting to look at the algorithm's parallel efficiency. This is defined as

$$E(P) = \frac{t_s}{t_P \cdot P}, \quad (15)$$

An efficiency of 100%, i.e. $E(P) = 1$, is achieved when all the processors are being used on the computation at all times and $S(P) = P$. For example, say that the best possible sequential algorithm runs in $t_s = 100$ seconds and that it takes $t_2 = 50$ seconds for two processors to run the parallel algorithm. This would give

$$S(2) = \frac{100}{50} = 2, \quad E(2) = \frac{100}{50 \cdot 2} = 1.$$

This gives a speedup of 2 and efficiency of 100%. In contrast, say that it takes 60 seconds for the two-processor algorithm to finish, this would give

$$S(2) = \frac{100}{60} \approx 1.67, \quad E(2) = \frac{100}{60 \cdot 2} \approx 0.83.$$

The lower speedup indicates that the parallel algorithm has some overhead. It also shows that only around 83% of the time is spent on the actual computations, the rest of the time is spent on communication or other overhead.

3.4 Hardware and Software

The Lattice BGK Boltzmann method was implemented in C++ and run on the supercomputer Dardel. It is part of the PDC Center for High Performance Computing at KTH Royal Institute of Technology. For the simulations in this report, only one out of Dardel's 1278 nodes was used, the number of cores used on that node ranged from the lowest possible one core, to the highest possible on a single node, 128 cores in parallel.

For visualizing and analyzing the results, the widely used numerical computing program Matlab was used to produce the plots for the two dimensional cases. For the more complicated three-dimensional case, the software Paraview was used. Paraview is one of the world's leading open source post-processing visualization engine, and can take a list of vectors as an input and display it in a 3D environment that lets the user pan and rotate the scene to better visualize it.

4 Numerical setup

This section describes the numerical setup of the project. Details about the different lattice structures used can be found in Section 4.1, and the initial conditions used for the two- and three-dimensional cases can be found in Section 4.2. 4.3 introduces the methods for validating the solutions as well as how to measure the error.

4.1 Lattice structures

The convergence tests were carried out for three different lattices in two dimensions and one lattice in three dimensions. Following the mathematical notation introduced in Section 2.1.2, the different lattices were D2Q7, D2Q9, D2Q13, and D3Q19. Table 4.1 summarizes the properties of these lattices. For easier visualization, Figure 4.1 pictures the D2Q9 lattice, and figures of the other lattice structures can be found in Appendix A. For more details, see Xia et al. [23] (for D2Q13), and Krüger et al. [12] (for D2Q9 and D3Q19). One can check that all these lattice structures satisfy the isotropic condition given by the criteria (4).

Lattice	Velocities v_i	Weight w_i	c_s^2
D2Q7	(0,0)	1/2	1/4
	$\left(\cos \frac{2\pi j}{6}, \sin \frac{2\pi j}{6}\right), j = 1, \dots, 6$	1/12	
D2Q9	(0, 0)	4/9	1/3
	($\pm 1, 0$), ($0, \pm 1$)	1/9	
	($\pm 1, \pm 1$)	1/36	
D2Q13	(0,0)	2/5	2/5
	($\pm 1, 0$), ($0, \pm 1$)	8/75	
	($\pm 1, \pm 1$)	1/25	
	($\pm 2, 0$), ($0, \pm 2$)	1/300	
D3Q19	(0,0,0)	1/3	1/3
	($\pm 1, 0, 0$), ($0, \pm 1, 0$), ($0, 0, \pm 1$)	1/18	
	($\pm 1, \pm 1, 0$), ($\pm 1, 0, \pm 1$), ($0, \pm 1, \pm 1$)	1/36	

Table 4.1: Table with the properties of each lattice structure. For every structure, denoted with the $DdQn$ notation, the speed of sound squared c_s^2 along with the list of the corresponding velocity vectors v_i paired with their respective weights w_i are presented.

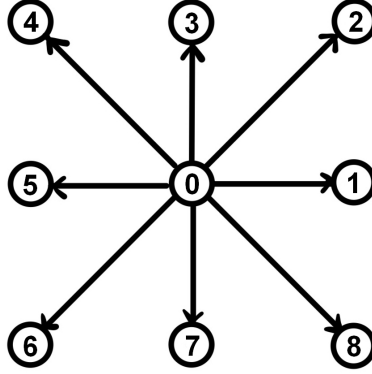


Figure 4.1: Visualization of the D2Q9 lattice structure, with four unit velocities in each cardinal direction, four diagonal velocities, and one velocity vector with all components zero.

4.2 Initial conditions

Since this project aims to expand on the work of Gu et al. [10] by investigating if the solution's convergence rate when approaching the hydrodynamic limit depends on the lattice structure chosen, the same two initial conditions in two dimensions will be investigated. These are given in terms of the vorticity $\omega(x, y)$ of the velocity field. The description of vorticity is given in Appendix B. The third initial condition is a turbulent randomly generated one. These initial conditions are described as follows:

- The Taylor-Green vortex

$$\omega(x, y, t = 0) = 10 \sin(4\pi x) \sin(4\pi y). \quad (16)$$

From Gu et al. [10], this initial condition has the exact solution

$$\omega(x, y, t) = \omega(x, y, t = 0) \cdot \exp(-32\pi^2 c_s^2 \nu t), \quad (17)$$

where c_s is the speed of sound associated with the lattice and ν is the kinematic viscosity.

- The perturbed Taylor-Green vortex

$$\omega(x, y, t = 0) = -\sin(2\pi x) \sin(2\pi y) + \exp\left(-50((x-1/2)^2 + (y-1/2)^2)\right) + C \quad (18)$$

where C is a constant chosen appropriately for the initial condition to satisfy the zero-mean condition (explained in Appendix C). This initial condition does not have an analytical solution, the method used to validate the results will be described in Section 4.3.

- To obtain a two-dimensional turbulent initial condition, random Fourier modes are generated and time-stepped using a Navier-Stokes solver until the solution is coherent enough to be used as an initial condition to the Boltzmann solver. This ensures smoothness, some randomness, and periodicity.

For three dimensional initial conditions, Antuono [1] has derived an exact solution expressed in the three velocity component ($\mathbf{u} = (u, v, w)$):

$$\mathbf{u}(x, y, z, t) = \begin{cases} u = \left(A \sin(kz) + C \cos(ky) \right) e^{-c_s^2 \nu k^2 t} \\ v = \left(B \sin(kx) + A \cos(kz) \right) e^{-c_s^2 \nu k^2 t} \\ w = \left(C \sin(ky) + B \cos(kx) \right) e^{-c_s^2 \nu k^2 t} \end{cases}, \quad (19)$$

where A, B, C are arbitrary constants (A,B,C=1 in this report) and $k = 2\pi/L$ where L is the length of the domain. The initial condition is given by setting $t = 0$ in (19). Although this is not a fully three dimensional motion since the components of the velocity field do not depend on all the spatial variables, it is still a relevant test of the implementation and convergence rate.

For all the two dimensional cases in vorticity form and the three dimensional case in terms of the velocity field, the initial condition $\hat{\mathbf{g}}^\epsilon(\mathbf{x}, t = 0)$ needs to be computed. For the two dimensional case, this is done in the same way as Gu et al. [10] by computing the velocity vector $\mathbf{u}$ from the given vorticity initial condition (this step is explained in Appendix B) and inputting that into $\hat{\mathbf{g}}_{eq}^\epsilon(\mathbf{x}, t = 0)$, and finally setting $\hat{\mathbf{g}}^\epsilon(\mathbf{x}, t = 0) := \hat{\mathbf{g}}_{eq}^\epsilon(\mathbf{x}, t = 0)$. For the three dimensional case, this process is the same except for the first step that can be skipped since the initial condition already is given in terms of the velocity components.

Along with the initial conditions, an appropriate time step Δt has to be chosen for the time stepping described in Section 3.1.2. Since the computation time relies heavily on this constant, it is important to choose a value that is small enough to ensure numerical stability, but large enough for the implementation to terminate in reasonable time. This system of equations is so-called "stiff" in the sense that decreasing the Knudsen number means that Δt needs to decrease as well to ensure stability. To find an appropriate value for Δt depending on the chosen ϵ , a linear search for a stable time step can be done, decreasing Δt and restarting the simulation if the solution diverges. This way, a good balance between running time and numerical stability can be achieved.

After having chosen an appropriate time step Δt and having time stepped to the final time, the interesting quantities such as the velocity vector field and density scalar field can be recovered from $\mathbf{g}_n^\epsilon$ using Equation (7).

4.3 Validation and visualization

Validation is an important part of every numerical simulation. When no analytical solution exists, this process becomes even more challenging, as the reference solution must be obtained using proven and trusted software.

As mentioned in Section 4.2, initial conditions (16) has the exact solution at some time $t > 0$ given by (17). This makes it easy to validate the simulation results since the difference between the exact solution and the simulation can be computed. The same can be said for the initial solution given by (19), where the variable t can be set appropriately to give a reference solution for all time steps. For initial conditions (18) and the turbulent one however, no analytical solution exists.

Using the vorticity transport equation solver from Matharu, Protas, and Yoneda [14] with a small timestep, a reference solution was obtained. This is the same method as was used in Gu et al. [10] and the full implementation of the solver can be found at Matharu [13].

Now that reference solutions for all initial conditions considered are accessible, a measure of how similar they are is needed. Again similar to Gu et al. [10], the

normalized L^2 -norm of the difference between the simulation results ω^ϵ and the reference solution ω is used for the two-dimensional cases. This error can be defined as

$$\text{Error}(\epsilon) := \frac{\|\omega(\mathcal{T}) - \omega^\epsilon(\mathcal{T})\|_{L^2(\mathbf{T}_x^2)}}{\|\omega(\mathcal{T})\|_{L^2(\mathbf{T}_x^2)}},$$

where

$$\|\omega(\cdot, \cdot, t)\|_{L^2(\mathbf{T}_x^2)} = \sqrt{\sum_{i,j=0}^{N-1} \omega(x_i, y_j, t)^2 \cdot \Delta x \Delta y}.$$

The time $\mathcal{T}$ is the time at which the two solutions are compared. In this project, $\mathcal{T} = 1$ is used for initial condition (16) and $\mathcal{T} = 32$ for initial condition (18). The reason for the later time in the second case is to wait until the most interesting parts of the resulting flow have developed. This error can then be plotted against different values of ϵ to obtain a convergence rate.

For measuring the error of the three dimensional solution, there is no possibility to generate a vorticity scalar field to directly compare. To still be able to investigate the convergence of the solution, the following measure is used:

$$\text{Error}(\epsilon) := \frac{1}{3} \sum_{i=1}^3 \|u_i(\mathcal{T}) - u_i^\epsilon(\mathcal{T})\|_{L_\infty(\mathbf{T}_x^3)}$$

where $\mathbf{u} = (u_1, u_2, u_3)$ are the velocity components of the reference solution and $\mathbf{u}^\epsilon = (u_1^\epsilon, u_2^\epsilon, u_3^\epsilon)$ are the velocity components from the simulation. The notation $u_i(\mathcal{T})$ and $u_i^\epsilon(\mathcal{T})$ refers to the reference and simulation results at some time of interest $\mathcal{T} = 1$ for the three dimensional case. The discrete L_∞ norm of g over the domain $\mathbf{T}_x^3$ is defined as

$$\|g(\cdot, \cdot, \cdot, \mathcal{T})\|_{L_\infty(\mathbf{T}_x^3)} = \max_{0 \leq i,j,k \leq N-1} |g(x_i, y_j, z_k, \mathcal{T})|.$$

In words, the measure of error is the average maximum deviation from the reference solution across the three velocity components. In the same way as for the two dimensional case, this error can be plotted for different values of ϵ to investigate the convergence rate of the method.

4.4 Algorithm

This section provides the pseudo code describing the algorithm used. Algorithm 1 describes the RK4 procedure, Algorithm 2 describes the process of computing the RHS of the ODE (11).

The full code developed for this project and used to run all simulations can be found at [15].

Algorithm 1 Pseudo-code describing the RK4 algorithm

Require: Initial condition for $\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon$, a RHS function (see Algorithm 2), a final time $\mathcal{T}$, a time step Δt .

- 1: **for** $t_i = 1$ to $\mathcal{T}/\Delta t$ **do**
- 2: $\mathbf{s}_1 \leftarrow \text{RHS}(\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon)$
- 3: $\mathbf{s}_2 \leftarrow \text{RHS}(\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon + \mathbf{s}_1/2)$
- 4: $\mathbf{s}_3 \leftarrow \text{RHS}(\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon + \mathbf{s}_2/2)$
- 5: $\mathbf{s}_4 \leftarrow \text{RHS}(\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon + \mathbf{s}_3)$
- 6: $\hat{\mathbf{g}}_{eq,\mathbf{k}} \leftarrow \hat{\mathbf{g}}_{eq,\mathbf{k}} + \frac{\Delta t}{6}(\mathbf{s}_1 + 2\mathbf{s}_2 + 2\mathbf{s}_3 + \mathbf{s}_4)$
- 7: **end for**
- 8: $\mathbf{g}_{eq}^\epsilon \leftarrow$ Compute inverse FFT of $\hat{\mathbf{g}}_{eq,\mathbf{k}}$
- 9: $\rho^\epsilon, \mathbf{u}^\epsilon \leftarrow$ Use (7)

Ensure: Final solution $\rho^\epsilon(\mathcal{T}, \mathbf{x}), \mathbf{u}^\epsilon(\mathcal{T}, \mathbf{x})$

Algorithm 2 Pseudo-code describing the computation of (11).

- 1: **function** $\text{RHS}(\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon)$
 - 2: Compute inverse FFT of $\hat{\mathbf{g}}_{eq,\mathbf{k}}^\epsilon$ to get $\mathbf{g}_{eq}^\epsilon$
 - 3: $\hat{\rho}_{\mathbf{k}}^\epsilon, \hat{\mathbf{u}}_{\mathbf{k}}^\epsilon \leftarrow$ Compute using (7)
 - 4: Compute inverse FFT of $\hat{\mathbf{u}}_{\mathbf{k}}^\epsilon$.
 - 5: Compute non-linear term $u_\alpha^\epsilon u_\beta^\epsilon$ in physical space.
 - 6: Compute FFT of $u_\alpha^\epsilon u_\beta^\epsilon$, denoted by $(\widehat{u_\alpha^\epsilon u_\beta^\epsilon})_{\mathbf{k}}$.
 - 7: Use (13) to dealias $(\widehat{u_\alpha^\epsilon u_\beta^\epsilon})_{\mathbf{k}}$.
 - 8: Compute $\hat{g}_{i,eq,\mathbf{k}}^\epsilon$ (10) in Fourier space for $i = 0, \dots, n$.
 - 9: Compute the RHS in (11)
 - 10: **end function** RHS
-

5 Results

The result section is structured as follows: Section 5.1 presents visualization of the different initial conditions and snapshots of the simulation results in time. Section 5.2 presents the results regarding the hydrodynamic limit, firstly when changing lattice structures, secondly when running the simulations in three dimensions, and thirdly when running a turbulent initial condition with much larger spatial resolution and lower viscosity. Section 5.3 presents the running times and parallel speedup of the implementation under different conditions.

5.1 Visualization of results

In Figure 5.1, snapshots of the vorticity field are shown for times $t = 0, 8, 16, 32$ using the initial condition (18). A full animation of the flow's evolution can be found at [16]. Figure 5.2 shows the vorticity field of the initial condition (16). Since the solution is a decaying of the initial condition, there is no difference in the shape of the vorticity field for different times, thus only the field at $t = 0$ is shown. Figure 5.3 shows snapshots of the vorticity field at times $t = 0, 1, 2, 4$ using the turbulent initial condition discussed in Section 4.2. A full animation of this initial condition can be found at [17]. For the three dimensional case, Figure 5.4 shows the velocity vector $\mathbf{u} = (u, v, w)$ obtained from the initial condition (19). To better visualize this initial condition, Appendix E includes two-dimensional slices of the flow at $x = 0$ (Figure E.1), $y = 0$ (Figure E.2), and $z = 0$ (Figure E.3).

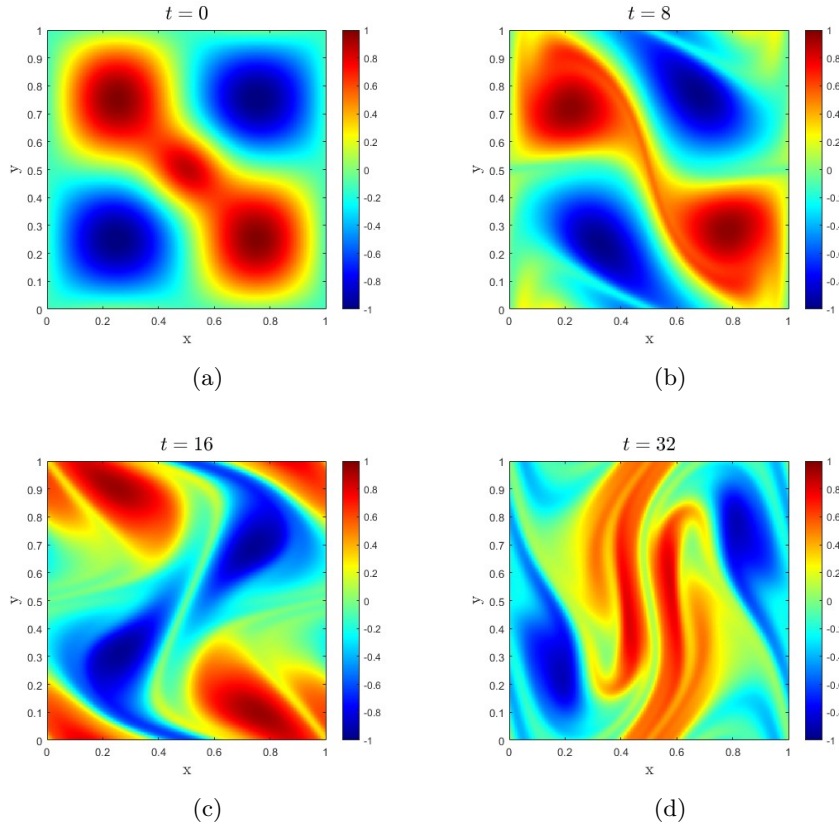


Figure 5.1: Snapshots of the vorticity field at times (a) $t = 0$, (b) $t = 8$, (c) $t = 16$, (d) $t = 32$, associated with the flow resulting from the initial condition (18). This figure matches Figure 1 in Gu et al. [10].

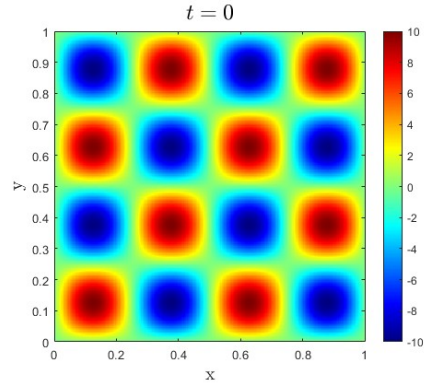


Figure 5.2: The initial condition (16) of the vorticity field.

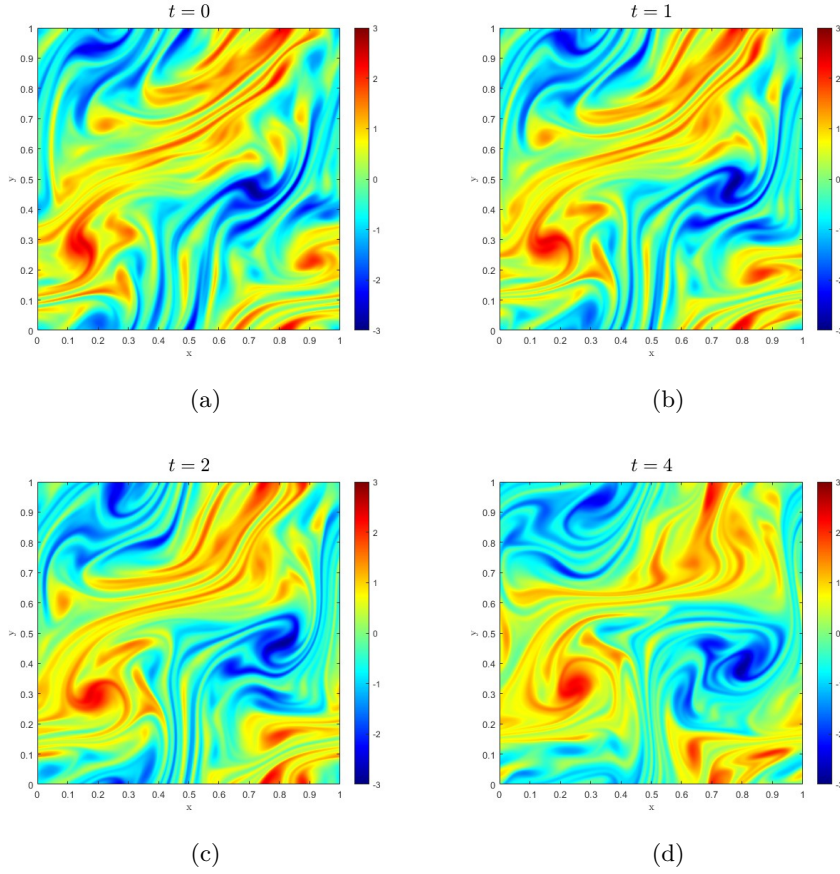


Figure 5.3: Snapshots of the vorticity field at times (a) $t = 0$, (b) $t = 1$, (c) $t = 2$, (d) $t = 4$, associated with the flow resulting from the turbulent initial condition described in Section 4.2.

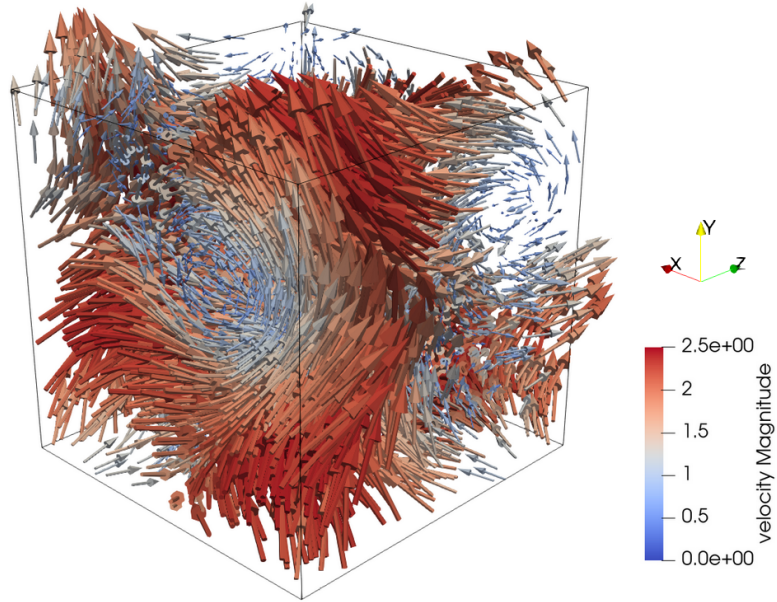


Figure 5.4: The initial condition (19) visualized by plotting the velocity vector at every grid point. The length and color of the arrows indicate the magnitude of the corresponding vector.

5.2 Hydrodynamic limit

This section presents the main results for this project, i.e. the convergence rates for the hydrodynamic limit. Section 5.2.1 displays how the convergence rate changes depending on the lattice used, Section 5.2.2 shows the results for the larger turbulent simulation, and Section 5.2.3 displays the convergence rate when running the simulations in three dimensions.

A summary of the choice of parameters for the different runs can be found in Table 5.1.

IC	Lattice	N	ν	$\mathcal{T}$	ϵ	Δt
TGV	D2Q7 D2Q9 D2Q13	128	10^{-4}	1.0	7.0	$2 \cdot 10^{-4}$
					1.0	$2 \cdot 10^{-4}$
					0.5	$2 \cdot 10^{-5}$
					0.1	$2 \cdot 10^{-6}$
					0.05	$2 \cdot 10^{-7}$
PTGV	D2Q7 D2Q9 D2Q13	128	10^{-4}	32.0	1.0	$2 \cdot 10^{-4}$
					0.5	$2 \cdot 10^{-5}$
					0.1	$2 \cdot 10^{-6}$
Turbulent	D2Q9	2048	$1.5 \cdot 10^{-5}$	4.0	1.0	$4 \cdot 10^{-5}$
					0.8	$2 \cdot 10^{-5}$
					0.5	$1 \cdot 10^{-5}$
3D IC	D3Q19	32	10^{-4}	1.0	1.0	$2 \cdot 10^{-4}$
					0.8	$2 \cdot 10^{-5}$
					0.5	$2 \cdot 10^{-5}$
					0.1	$2 \cdot 10^{-6}$

Table 5.1: Table containing the combinations of parameters for the different simulations (Initial condition, lattice structure, spatial resolution, kinematic viscosity, final time, Knudsen number and time step). The IC column includes TGV (Taylor-Green vortex, Equation (16)), PTGV (Perturbed Taylor-Green vortex, Equation (18)), the turbulent IC discussed in Section 4.2, and 3D IC (the three dimensional initial condition (19)). The rows in the table represent all possible permutations of parameters.

5.2.1 Lattice modification

Figures 5.5 and 5.6 present the convergence rate corresponding to the different lattice structures D2Q7, D2Q9 and D2Q13 measuring the error as discussed in Section 4.3, for the two-dimensional initial conditions (16) and (18), respectively. The convergence rates can in all cases be accurately described by a power-law relation obtained with a least-square fit of the form

$$\text{Error}(\epsilon) \approx a \cdot \epsilon^b, \quad (20)$$

where a and b are to be determined. Table 5.2 describes the parameters obtained by the least-square fit for initial condition (16) and (18), respectively. In all cases and except for the factor, very similar convergence rates were observed, essentially having an error decreasing with $\mathcal{O}(\epsilon^2)$ independent of the lattice structure used.

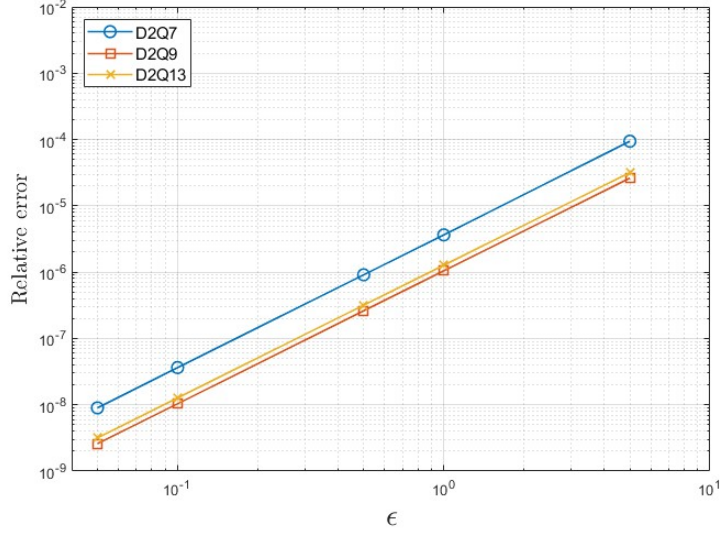


Figure 5.5: Dependence of the relative error on ϵ for the three different lattices D2Q7 (blue circles), D2Q9 (orange squares) and D2Q13 (yellow crosses) and the Taylor-Green vortex initial condition (16). The circles and crosses represents the resulting data points and the solid lines are the power-law fits.

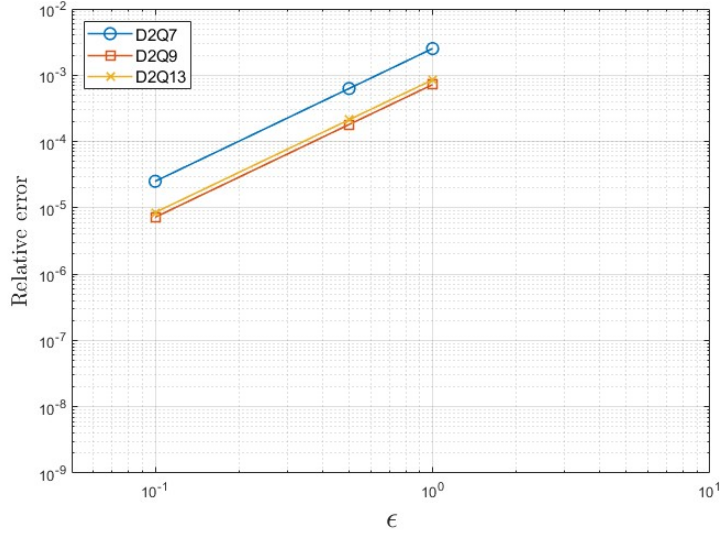


Figure 5.6: Dependence of the relative error on ϵ for the three different lattices D2Q7 (blue circles), D2Q9 (orange squares) and D2Q13 (yellow crosses) and the perturbed Taylor-Green vortex initial condition (18). The circles and crosses represents the resulting data points and the solid lines are the power-law fits.

Lattice	TGV	PTGV
D2Q7	$3.6861 \cdot 10^{-6} \cdot \epsilon^{2.0087}$	$2.5218 \cdot 10^{-3} \cdot \epsilon^{2.0012}$
D2Q9	$1.0454 \cdot 10^{-6} \cdot \epsilon^{2.0026}$	$7.2178 \cdot 10^{-4} \cdot \epsilon^{2.0001}$
D2Q13	$1.2709 \cdot 10^{-6} \cdot \epsilon^{2.0006}$	$8.5178 \cdot 10^{-4} \cdot \epsilon^{2.0002}$

Table 5.2: A description of the power-law fits for the different lattices and initial conditions. The TGV column represents initial condition (16) and Figure 5.5. The PTGV column represents initial condition (18) and Figure 5.6.

5.2.2 Turbulent simulation

For the turbulent initial condition described in Section 4.2, Figure 5.7 shows the measured errors as a function of the Knudsen number ϵ along with the power-law fit (20) given by

$$\text{Error}(\epsilon) \approx 5.2360 \cdot 10^{-4} \cdot \epsilon^{2.0019}. \quad (21)$$

This shows the same behavior as the analytically defined initial conditions and higher values for the viscosity in Section 5.2.1, i.e. that the error decreases with $\mathcal{O}(\epsilon^2)$ when ϵ goes to zero.

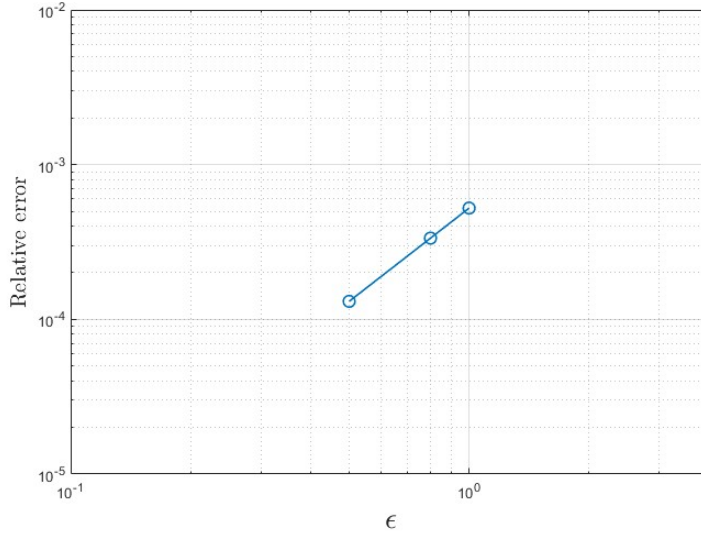


Figure 5.7: Dependence of the relative error on ϵ for the lattice structure D2Q9 and the turbulent initial condition described in Section 4.2. The circles represent the resulting data points and the solid line is the power-law fit (21).

5.2.3 Higher dimensions

Similarly to the two-dimensional cases, Figure 5.8 pictures the convergence rate for the three dimensional simulations. One can in the same way as previously see that the relation between the average L_∞ error discussed in Section 4.3 and ϵ corresponds to a

power law. A least-square fit results in the following relation:

$$\text{Error}(\epsilon) \approx 2.6262 \cdot 10^{-7} \cdot \epsilon^{2.0001}. \quad (22)$$

This follows the same convergence rate as for the two-dimensional cases, with the error decreasing as $\mathcal{O}(\epsilon^2)$.

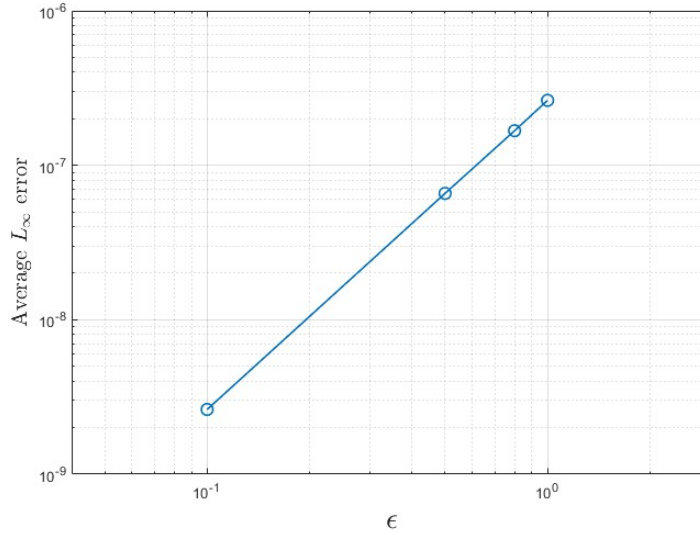


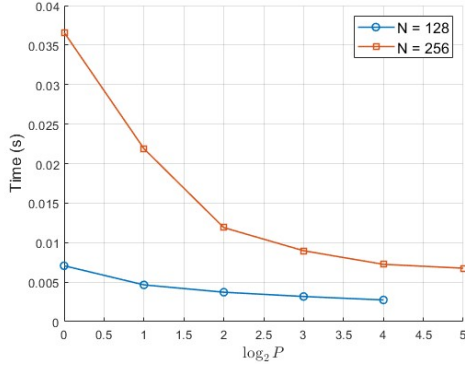
Figure 5.8: Dependence of the relative error on ϵ for the lattice structure D3Q19 and the initial condition (19). The circles represents the resulting data points and the solid line is the power-law fit (22).

5.3 Parallel performance

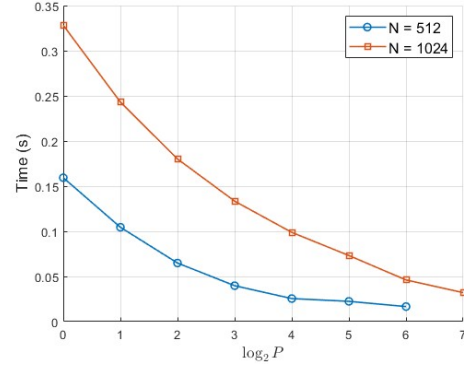
Evaluating the parallel performance by timing the simulation runs using a varying amount of processors, the results for the two dimensional implementation are shown in Figure 5.9 with the exact measurements in Appendix F, Table F.1. A clear increase in speed is seen when increasing the number of processes across all choices of spatial resolution.

Similarly, the timings for the three dimensional runs can be found in Figure 5.10. The exact times can be found in Table F.2, Appendix F. The same behavior can be seen there, with an increase in speed for every increase in the number of processes.

The speedup and efficiency as defined in Equations (14) and (15) are presented in Table 5.3 for the two-dimensional case with $N = 1024$. The speedup is increasing up when increasing the number of processors, as seen in Figure 5.9. However, the efficiency goes down dramatically from around 68% with two processors, all the way down to 8% for 128 processors. This indicates that although the runtime still decreases with more processors, the gain for each additional processor becomes progressively smaller, to the point that only about 8% of the algorithm's time is actually used for computations and the rest for communication or other overhead operations.



(a)



(b)

Figure 5.9: Visualization of the time needed per time step for the two-dimensional implementation as a function of the number of processors, P . Figure (a) shows $N = 128$ in blue circles and $N = 256$ in orange squares. Figure (b) shows $N = 512$ in blue circles and $N = 1024$ in orange squares.

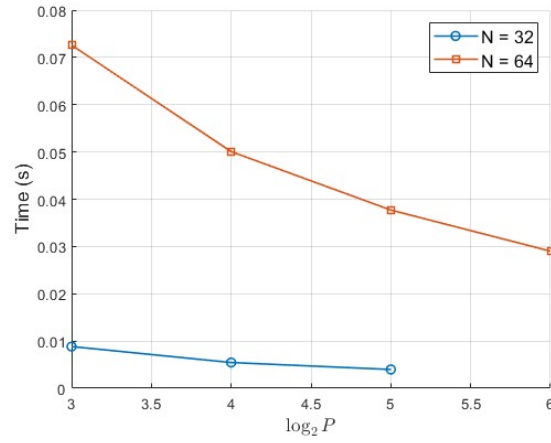


Figure 5.10: Visualization of the time needed per time step for the three-dimensional implementation as a function of the number of processors, P , for $N = 32$ in blue circles, and $N = 64$ in orange squares.

P	Speedup $S(P)$	Efficiency $E(P)$
1	-	-
2	1.350	0.675
4	1.823	0.456
8	2.460	0.308
16	3.322	0.208
32	4.484	0.140
64	7.113	0.111
128	10.228	0.080

Table 5.3: Summary of the speedup and efficiency (Equations (14) and (15)) as functions of the number of processors P , for the two-dimensional implementation in the case $N = 1024$.

6 Discussion

6.1 Lattice modification

The convergence for the D2Q9 lattice correspond very well with the results by Gu et al. [10], demonstrating that the implementation of the base algorithm is done correctly.

Changing lattice structure affects the results in multiple ways. Firstly, for both the two-dimensional initial conditions, the value of the constants in the fitted error relations are higher when having a simpler lattice with fewer velocity vectors like the D2Q7 lattice. This indicates that less complex lattice structures do not result in the same accuracy for a given value of the Knudsen number. The difference between the D2Q9 and D2Q13 is relatively small, indicating that the complexity of the D2Q9 lattice is enough to get a good enough accuracy. Interestingly, even though the actual constants in the error differ when changing the initial condition, the relationship between the different lattices stay the same, suggesting that this behavior does not depend on the initial conditions chosen.

It is important to mention that since this project focuses on the limit itself, i.e. when $\epsilon \rightarrow 0$, the exact value of the constant term is irrelevant since the value of the error is dominated by ϵ^2 .

6.2 Turbulent simulation

The convergence tests for the turbulent simulation show similar results to those for the lattice modifications, i.e. that the error decreases as $\mathcal{O}(\epsilon^2)$. As mentioned, the constant term is not important since the interesting part is the limit when $\epsilon \rightarrow 0$. This convergence rate is especially interesting since the resulting flow is much more complex than the analytically defined initial conditions. In addition, this confirms that the convergence is behaving as expected for lower values of the relaxation time ν .

6.3 Higher dimensions

When running the three-dimensional simulations, the relationship between the numerical error and the Knudsen number once again clearly shows a power-law relationship proportional to the square of the Knudsen number, as shown in Equation (22). This shows that the simulations taking place in two or three dimensions, does not change the convergence rate. In the same way as for two dimensions, the constant term in the power-law relation is irrelevant since the value for the error is dominated by ϵ^2 when $\epsilon \rightarrow 0$.

Due to numerical instabilities produced by floating point inaccuracies, the three-dimensional simulations could only be run for some time before diverging. This time depended on ϵ , with lower values producing longer stable solutions. A rough estimate of the numerical error arising from floating point inaccuracies can be made from the precision of a double-precision variable, $\sim 10^{-15}$, the number of discrete grid points, 64^3 , and the number of time steps taken before observing a diverging solution, ~ 160000 . The error would then be approximately $4.2 \cdot 10^{-5}$, only increasing for higher spatial resolution and simulation time. This is a large error that is enough to compromise the stability of the system.

6.4 Running time

When timing the different implementations and measuring their parallel performances, it is clear that the algorithm is faster when increasing the number of processors. This is expected since each process only handles a fraction of the physical grid and thus needs to do less work than if one processor was to operate on everything. The Fast Fourier transform is also expected to be faster in a parallel environment.

Looking at the speedup and efficiency in Table 5.3. It can be seen that the speedup is increasing but efficiency is decreasing as more processors are used. This is also expected

since communication increases when using more processors. Since the Fast Fourier transform operations are the dominating part of the algorithm, and these operations all need two all-to-all communication steps, it is expected that these communications would take a long time compared to the element-wise operations that do not need any communication. If the spatial dimension for the efficiency test was increased so that more time was actually spent on the computations themselves instead of communication, the efficiency of the program should be higher. With $N = 1024$ and $P = 128$ (the last row of Table 5.3), each processors only hold a group of eight columns in the grid. This means that the actual computation is very fast, but the communications are many and bottlenecks the program.

7 Conclusion and future work

In conclusion, the implementations show promising results. The parallelized two-dimensional algorithm is fast enough to run larger, longer, and more turbulent simulations within a reasonable time frame. Furthermore, numerically verifying that the convergence rate does not depend on the lattice structure used opens up the possibility for a general mathematical proof on the convergence rate of the method independent from the exact lattice structure used.

The results from the three-dimensional implementation are also promising. Having an implementation that is fast enough to run meaningful simulations with, opens up possibilities to investigate other initial conditions, boundary conditions, and convergence criteria that previously were largely confined to two dimensional problems. The convergence rate being the same as for the two-dimensional case is also interesting since it opens up the possibility of finding some general mathematical expression for the convergence rate of the velocity-discretized LBGK Boltzmann method that is independent of both dimension and lattice structure.

7.1 Future work

The main area of improvement is the implementation of bounded domains using the lattice BGK Boltzmann method. This is important for using the method in real world applications. This is a challenge since accurately enforcing boundary conditions such as no-slip walls, inflow/outflow boundaries, and moving solid surfaces requires other methods than the Fourier-bases that this report has been built upon. It becomes even more complicated when having to consider curved boundaries.

Two possibilities for handling boundary conditions are the bounce-back and wet-node approaches. The bounce-back method is possibly the more used because of its simplicity, although it struggles with accuracy near boundaries with more complex geometry. It enforces a no-slip condition by reflecting particle distribution functions at the boundary. In contrast, wet-node boundary conditions has higher accuracy, and involve interpolation to estimate the distribution functions at the boundaries. More about these methods can be found in [12].

A continuation of this project could therefore be to investigate the error at the hydrodynamic limit when having implemented these boundary conditions. One could also compare them in different scenarios, as well as implementing them in both two and three dimensions.

There are also improvements to be made regarding the speed of the implementations. For example, none of the element-wise operations in the implementation are vectorized. Vectorizing the code using an efficient and optimized method could lead to increases in speed.

References

- [1] Antuono, Matteo. “Tri-periodic fully three-dimensional analytic solutions for the Navier–Stokes equations”. In: *Journal of Fluid Mechanics* 890 (May 2020), A23. DOI: 10.1017/jfm.2020.126.
- [2] Boyd, John P. *Chebyshev & Fourier Spectral Methods*. 1989.
- [3] Canuto, Claudio, Hussaini, M. Yousuff, Quarteroni, Alfio, and Zang, Thomas A. *Spectral Methods in Fluid Dynamics*. Berlin, Heidelberg: Springer, 1988. ISBN: 978-3-540-52205-8 978-3-642-84108-8. DOI: 10.1007/978-3-642-84108-8.
- [4] Chu, Eleanor and George, Alan. *Inside the FFT Black Box: Serial and Parallel Fast Fourier Transform Algorithms*. Vol. 1. Computational Mathematics. CRC Press, Nov. 11, 1999. ISBN: 978-0-8493-0270-1 978-1-4200-4996-1. DOI: 10.1201/9781420049961.
- [5] Dutykh, Denys. “A brief introduction to pseudo-spectral methods: application to diffusion problems”. In: 2019, pp. 167–209. DOI: 10.1007/978-3-030-31574-0_8. arXiv: 1606.05432[math].
- [6] Evans, Lawrance C. *Partial Differential Equations*. 19 vols. Graduate Studies in Mathematics. American Mathematical Society, 1997.
- [7] Fefferman, Charles L. “Existence and smoothness of the navier stokes equations”. In: (2000).
- [8] *FFTW docs*. URL: <https://www.fftw.org/> (visited on 04/03/2025).
- [9] Galdi, Giovanni P. “An Introduction to the Mathematical Theory of the Navier-Stokes Equations”. In: (2011).
- [10] Gu, Zhongyang, Hu, Xin, Matharu, Pritpal, Protas, Bartosz, Sasada, Makiko, and Yoneda, Tsuyoshi. “The incompressible Navier–Stokes limit from the discrete-velocity BGK Boltzmann equation”. In: *Nonlinearity* 38.5 (Apr. 2025). Publisher: IOP Publishing, p. 055014. ISSN: 0951-7715. DOI: 10.1088/1361-6544/adca81.
- [11] Hou, Thomas Y. and Li, Ruo. “Computing Nearly Singular Solutions Using Pseudo-Spectral Methods”. In: *Journal of Computational Physics* 226.1 (Sept. 2007), pp. 379–397. ISSN: 00219991. DOI: 10.1016/j.jcp.2007.04.014. arXiv: math/0701337.
- [12] Krüger, Timm, Kusumaatmaja, Halim, Kuzmin, Alexandr, Shardt, Orest, Silva, Goncalo, and Viggren, Erlend Magnus. *The Lattice Boltzmann Method: Principles and Practice*. Graduate Texts in Physics. Cham: Springer International Publishing, 2017. ISBN: 978-3-319-44647-9 978-3-319-44649-3. DOI: 10.1007/978-3-319-44649-3.

- [13] Matharu, Pritpal. *Boltz2NS*. <https://github.com/pipmath/Boltz2NS>. Dec. 18, 2024.
- [14] Matharu, Pritpal, Protas, Bartosz, and Yoneda, Tsuyoshi. “On maximum enstrophy dissipation in 2D Navier–Stokes flows in the limit of vanishing viscosity”. In: *Physica D: Nonlinear Phenomena* 441 (Dec. 2022), p. 133517. ISSN: 01672789. DOI: 10.1016/j.physd.2022.133517.
- [15] Monpouet Ekeram, Théo. *Discrete Velocity BGK Boltzmann Github*. <https://github.com/TheoMonpouet/DiscreteVelocityBGKBoltzmann2025>. June 20, 2025.
- [16] Monpouet Ekeram, Théo. *Discrete-velocity BGK Boltzmann method - Perturbed Taylor-Green Vortex initial condition*. https://youtu.be/O_3QnU_v1YM. May 27, 2025.
- [17] Monpouet Ekeram, Théo. *Discrete-velocity BGK Boltzmann method - Turbulent initial condition*. <https://youtu.be/mJ6kkXU2TcM>. May 27, 2025.
- [18] Richardson, Lewis Fry. *Weather prediction by numerical process*. In collab. with University of California Libraries. Cambridge, The University press, 1922. 262 pp.
- [19] Saint-Raymond, L. “From the BGK model to the Navier–Stokes equations”. In: *Annales Scientifiques de l’École Normale Supérieure* 36.2 (Mar. 2003), pp. 271–317. ISSN: 00129593. DOI: 10.1016/S0012-9593(03)00010-7.
- [20] Sauer, Tim. *Numerical analysis*. 2nd ed. OCLC: 725295545. Boston: Pearson, 2012. ISBN: 978-0-321-78367-7.
- [21] Versteeg, H K and Malalasekera, W. *An Introduction to Computational Fluid Dynamics*. 2007. ISBN: 978-0-13-127498-3.
- [22] Wilkinson, Barry and Allen, Michael. *Parallel programming : techniques and applications using networked workstations and parallel computers*. 2. ed. Publication Title: Parallel programming : techniques and applications using networked workstations and parallel computers. Upper Saddle River, N.J: Pearson Prentice Hall, 2005. ISBN: 0-13-191865-6.
- [23] Xia, Muming, Zhou, Hui, Jiang, Chuntao, Cui, Jinming, Zeng, Yong, and Chen, Hanming. “Comparative Study of 2D Lattice Boltzmann Models for Simulating Seismic Waves”. In: *Remote Sensing* 16.2 (Jan. 2024). Number: 2 Publisher: Multidisciplinary Digital Publishing Institute, p. 285. ISSN: 2072-4292. DOI: 10.3390/rs16020285.

A Lattice structure visualization

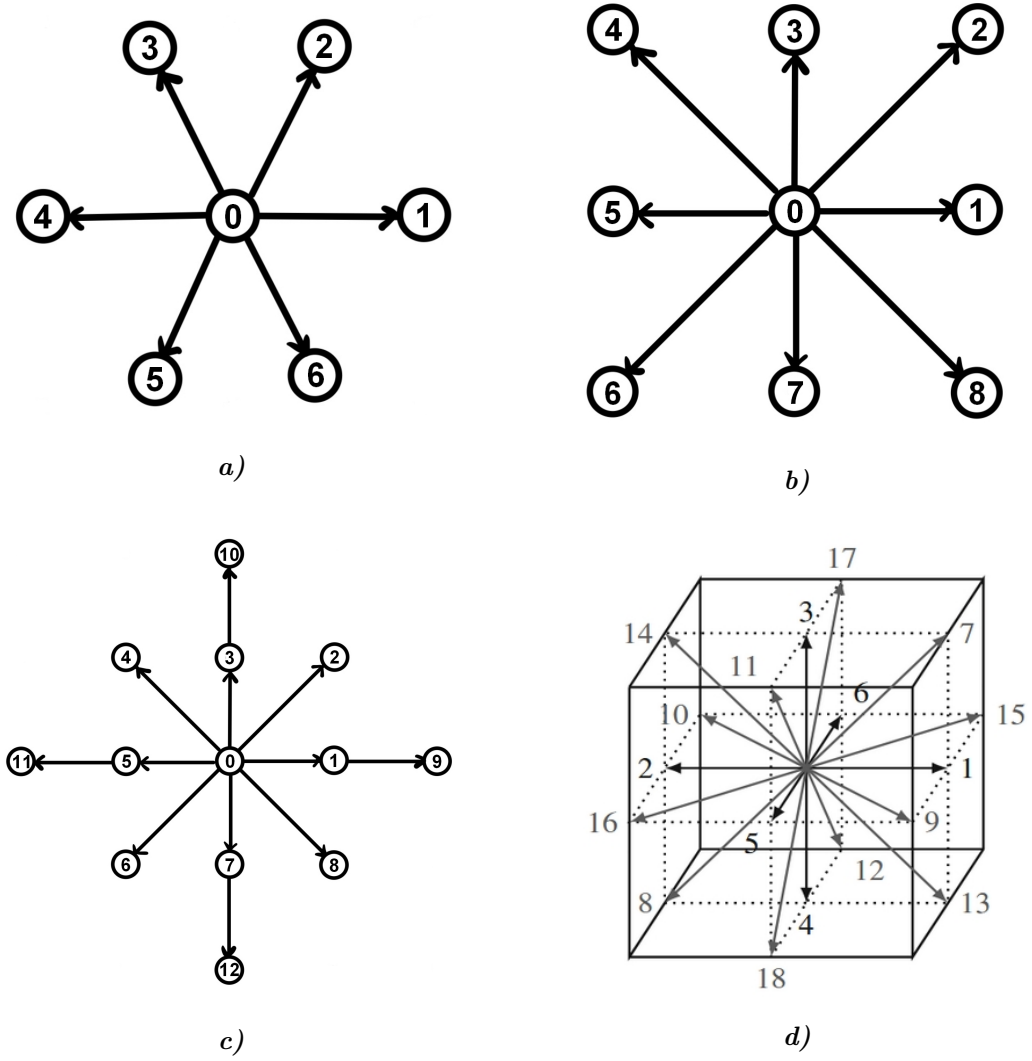


Figure A.1: Visualizations of lattices a) D2Q7 b) D2Q9 c) D2Q13 d) D3Q19 (Figure from Krüger et al. [12]).

B Vorticity description

The problem of being able to more conveniently visualize and compare two-dimensional vector fields can be solved using a vorticity scalar field. This way, a grid of vectors with two components can be turned into a grid of scalar values that is much easier to plot. Of course, the inverse operation can also be done (if the zero-mean condition described in Appendix C is satisfied) and it turns out that this is very simple when both the velocity and vorticity fields are handled in Fourier space.

B.1 From velocity to vorticity field

Going from the velocity field to the vorticity field is done as follows: let the vorticity ω of a two dimensional vector field $\mathbf{u} = (u, v)$ be defined as

$$\omega := \nabla_x^\perp \cdot \mathbf{u}, \quad \nabla_x^\perp := (-\partial_y, \partial_x).$$

More specifically, this results in

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}. \quad (23)$$

B.2 From vorticity to velocity field

Going from the vorticity scalar field to the velocity vector field is done by introducing the stream function $\psi(x, y)$ defined by

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}, \quad (24)$$

where $\mathbf{u} = (u, v)$. Plugging this into the definition of vorticity as given in (23) gives

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} = -\nabla^2 \psi. \quad (25)$$

Taking the Fourier transform of (25) gives

$$-(k_x^2 + k_y^2)\hat{\psi}_{k_x, k_y} = -\hat{\omega}_{k_x, k_y},$$

$$\hat{\psi}_{k_x, k_y} = \frac{\hat{\omega}_{k_x, k_y}}{k_x^2 + k_y^2}.$$

where k_x and k_y are the wave numbers in x and y directions. Equation (24) in Fourier space now gives

$$\begin{cases} \hat{u}_{k_x, k_y} = ik_y \hat{\psi}_{k_x, k_y} \\ \hat{v}_{k_x, k_y} = -ik_x \hat{\psi}_{k_x, k_y} \end{cases} = \begin{cases} \hat{u}_{k_x, k_y} = ik_y \frac{\hat{\omega}_{k_x, k_y}}{k_x^2 + k_y^2} \\ \hat{v}_{k_x, k_y} = -ik_x \frac{\hat{\omega}_{k_x, k_y}}{k_x^2 + k_y^2} \end{cases}.$$

From this, the inverse Fourier transform can be used to get u and v and reconstruct $\mathbf{u}$. For the case $k_x = k_y = 0$, the denominator in the expression for $\hat{\psi}_{k_x, k_y}$ is 0 and thus it becomes undefined. Set therefore $\psi_{0,0} = 0$. One could also achieve the same result using Biot-Savart's law, which is a way to invert a curl operation, but the stream function approach is the simplest for the two-dimensional case.

C Navier-Stokes in vorticity form

Using the same notation as Gu et al. [10], the Navier-Stokes equations can be written as

$$\begin{cases} \partial_t u - c_s^2 \nu \Delta_x u + \nabla_x \cdot (u \otimes u) + \nabla_x p = 0, \\ \nabla_x \cdot u = 0, \\ u(x, 0) = \mathbb{P}(u_0). \end{cases} \quad (26)$$

This can be simplified using the vorticity discussed in Appendix B. Applying the vorticity operator (23) to (26) gives

$$\begin{cases} \partial_t \omega + (u \cdot \nabla_x) \omega - c_s^2 \nu \Delta_x \omega = 0, \\ u = \nabla_x^\perp \Delta_x^{-1} \omega, \\ \omega(x, 0) = \nabla_x^\perp \cdot u_0, \end{cases} \quad (27)$$

where Δ_x^{-1} is the inverse Laplacian operator. To make sure that this operator is always invertible, the initial condition needs to satisfy the so-called zero-mean condition

$$\int_{T_x^2} \omega_0(x) dx = 0.$$

Since the mean vorticity in solutions of the 2D Navier-Stokes system is conserved, this condition means that the Laplacian will always be invertible.

D Orthogonality condition for PS method

For deriving the system of ordinary differential equations in Fourier space needed for the time stepping with the pseudo spectral method, an orthogonality condition is applied to the Fourier expansion of the PDE in question. This appendix will showcase an extension to the example found in the first chapter of Canuto et al. [3], where the orthogonality condition for the one-dimensional wave equation is presented. Here, the extension will include a more general second order PDE in two dimensions, in a way that is easily generalized for higher dimensions and other PDEs.

Let $u(x, y, t)$ be the objective function defined on the periodic square domain $(x, y) \in [-1, 1]^2$ equipped with the initial condition $u(x, y, 0) = f(x, y)$ that is also periodic on the domain. Let the PDE be defined as

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} + b \frac{\partial u}{\partial y} - c \frac{\partial^2 u}{\partial x^2} - d \frac{\partial^2 u}{\partial y^2} = 0, \quad (28)$$

where a, b, c, d are constants. As seen in Section 3.1.1, assuming that the spatial resolution is the same in both dimensions, i.e. $N_x = N_y = N$, the truncated Fourier series approximation of u , denoted u_K and its derivatives are given by

$$\begin{aligned} u_K(x, y, t) &= \sum_{k_x=-N/2}^{N/2-1} \sum_{k_y=-N/2}^{N/2-1} \hat{u}_{k_x, k_y}(t) e^{i\pi(k_x x + k_y y)}, \\ \frac{\partial u_K}{\partial t} &= \sum_{k_x, k_y} \frac{d}{dt} \hat{u}_{k_x, k_y}(t) e^{i\pi(k_x x + k_y y)}, \\ \frac{\partial u_K}{\partial x} &= \sum_{k_x, k_y} \hat{u}_{k_x, k_y}(t) (i\pi k_x) e^{i\pi(k_x x + k_y y)}, \quad \frac{\partial u_K}{\partial y} = \sum_{k_x, k_y} \hat{u}_{k_x, k_y}(t) (i\pi k_y) e^{i\pi(k_x x + k_y y)}, \\ \frac{\partial^2 u_K}{\partial x^2} &= \sum_{k_x, k_y} -(\pi k_x)^2 \hat{u}_{k_x, k_y}(t) e^{i\pi(k_x x + k_y y)}, \quad \frac{\partial^2 u_K}{\partial y^2} = \sum_{k_x, k_y} -(\pi k_y)^2 \hat{u}_{k_x, k_y}(t) e^{i\pi(k_x x + k_y y)}, \end{aligned}$$

Substituting these representations into the PDE (28) results in

$$\sum_{k_x, k_y} \underbrace{\left[\frac{d}{dt} \hat{u}(t) + a i\pi k_x \hat{u}(t) + b i\pi k_y \hat{u}(t) + c (\pi k_x)^2 \hat{u}(t) + d (\pi k_y)^2 \hat{u}(t) \right]}_{=: A(k_x, k_y, t)} e^{i\pi(k_x x + k_y y)} = 0.$$

The next step is applying the orthogonality condition in two dimensions. Multiply both sides of the equation by $e^{-i\pi(m x + n y)}$ where m, n are some integers and integrate then over the domain $[-1, 1]^2$.

$$\sum_{k_x, k_y} A(k_x, k_y, t) \cdot \int_{-1}^1 \int_{-1}^1 e^{i\pi(k_x x + k_y y - m x - n y)} dx dy = 0. \quad (29)$$

Using orthogonality to get the value of the integral part

$$\int_{-1}^1 \int_{-1}^1 e^{i\pi(k_x x + k_y y - m x - n y)} dx dy = 4\delta_{k_x, m} \delta_{k_y, n},$$

which, when substituting back into equation (29) gives

$$\sum_{k_x, k_y} A(k_x, k_y, t) 4\delta_{k_x, m} \delta_{k_y, n} = 0 \quad \Leftrightarrow \quad A(m, n, t) = 0.$$

Inserting the expression for $A(m, n, t)$, the final decoupled system of ODE:s to be time stepped is

$$\frac{d}{dt}\hat{u}_{m,n}(t) + ai\pi m\hat{u}_{m,n}(t) + bi\pi n\hat{u}_{m,n}(t) + c(\pi m)^2\hat{u}_{m,n}(t) + d(\pi n)^2\hat{u}_{m,n}(t) = 0.$$

For three dimensions, the process is the same except for using three dimensional Fourier series, and a three dimensional orthogonality condition. The process is also the same for the PDE given by (5).

E Slices of the three-dimensional vector field

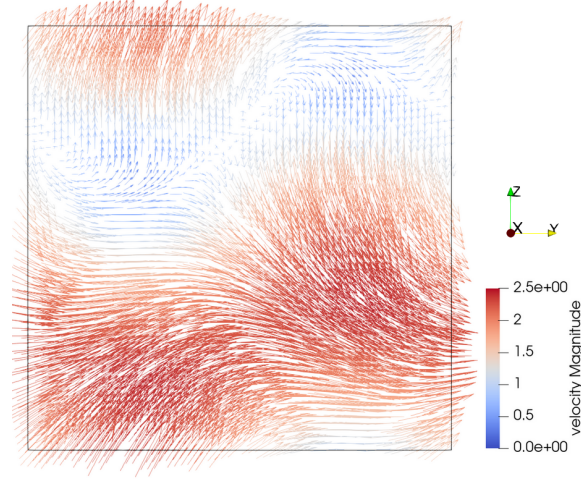


Figure E.1: The initial condition (19) as a two-dimensional slice along $x = 0$. The length and color of the arrows indicate the magnitude of the corresponding vector.

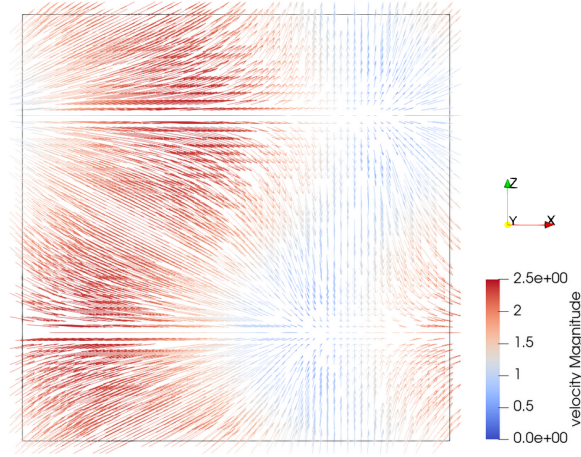


Figure E.2: The initial condition (19) as a two-dimensional slice along $y = 0$. The length and color of the arrows indicate the magnitude of the corresponding vector.

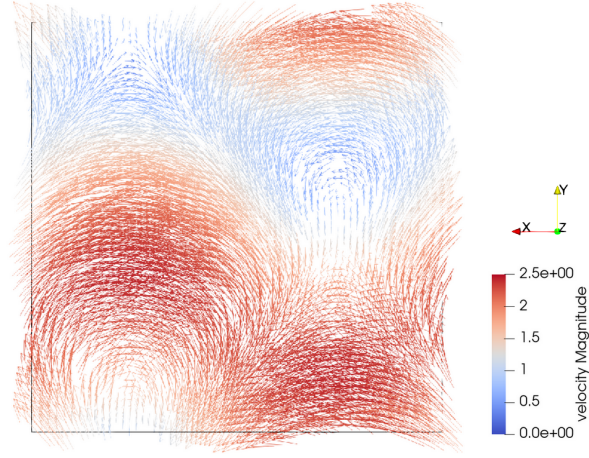


Figure E.3: The initial condition (19) as a two-dimensional slice along $z = 0$. The length and color of the arrows indicate the magnitude of the corresponding vector.

F Parallel performance tables

N	P	Time/timestep [s]
128	1	0.00707
	2	0.00465
	4	0.00372
	8	0.00318
	16	0.00274
256	1	0.03660
	2	0.02188
	4	0.01190
	8	0.00897
	16	0.00725
	32	0.00676

N	P	Time/timestep [s]
512	1	0.15037
	2	0.10448
	4	0.06486
	8	0.03978
	16	0.02552
	32	0.02245
1024	64	0.01667
	1	0.32820
	2	0.24311
	4	0.18008
	8	0.13340
	16	0.09881
	32	0.07319
	64	0.04614
	128	0.03209

Table F.1: Summary of the time needed per time step for the two-dimensional implementation. Results for $N = 128, 256, 512, 1024$ on 1-128 processors.

N	P	Time/timestep [s]
32	8	0.00883
	16	0.00545
	32	0.00398
64	8	0.07261
	16	0.05007
	32	0.03771
	64	0.02901

Table F.2: Summary of the time needed per time step for the three-dimensional implementation. Results for $N = 32, 64$ on 8-64 processors.

€€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Monpouet Ekeram",
    "First name": "Théo",
    "Local User Id": "tcame",
    "E-mail": "tcame@kth.se",
    "organisation": { "L1": "",
      },
    },
  "Cycle": "2",
  "Course code": "SF250X",
  "Credits": "30.0",
  "Degree1": { "Educational program": "Master's Programme, Applied and Computational Mathematics, 120 credits"
    , "programcode": "TTMAM"
    , "Degree": "Degree of Master of Science in Engineering"
    , "subjectArea": "Computational Mathematics"
    },
  "Title": {
    "Main title": "Parallel implementation and analysis of the discrete-velocity BGK Boltzmann method",
    "Language": "eng" },
  "Alternative title": {
    "Main title": "Parallel implementering och analys av den diskret-hastighetsbaserade BGK-Boltzmannmetoden",
    "Language": "swe"
  },
  "Supervisor1": { "Last name": "Matharu",
    "First name": "Pritpal",
    "E-mail": "pritpal.matharu@mis.mpg.de",
    },
  "Supervisor2": { "Last name": "Zahedi",
    "First name": "Sara",
    "E-mail": "sara.zahedi@math.kth.se",
    "organisation": { "L1": "",
      "L2": "Department of Mathematics" }
    },
  "Examiner1": { "Last name": "Zahedi",
    "First name": "Sara",
    "E-mail": "sara.zahedi@math.kth.se",
    "organisation": { "L1": "",
      "L2": "Department of Mathematics" }
    },
  "National Subject Categories": "10105",
  "Other information": { "Year": "2025", "Number of pages": "??,iii" },
  "Copyrightleft": "copyright",
  "Series": { "Title of series": "TRITA – " },
  "Opponents": { "Name": "Bernd-Roland Marcus Rijn" },
  "Presentation": { "Date": "2025-06-10 10:15"
    , "Language": "eng"
    , "Room": "3418"
    , "Address": "Lindstedtsvägen 25"
    , "City": "Stockholm" },
  "Number of lang instances": "1",
  "Abstract[eng ]": €€€€
  €€€€,
  "Keywords[eng ]": €€€€
  €€€€,
}
```